

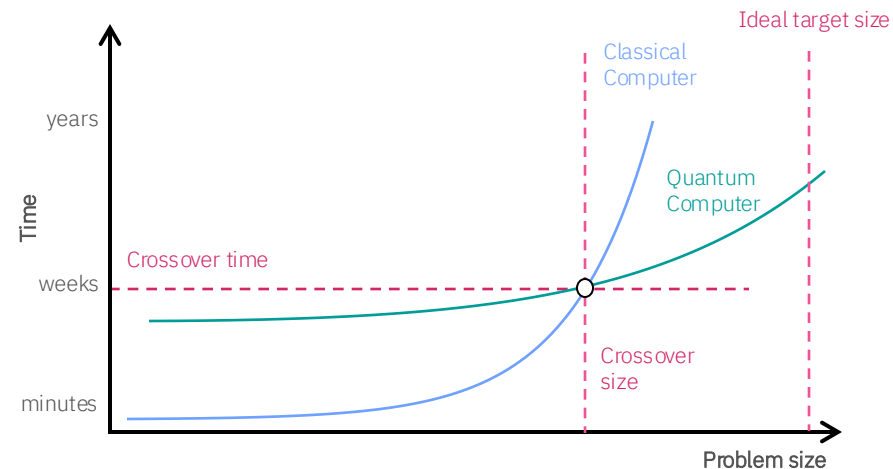
The Journey of Quantum Algorithms for Combinatorial Optimization

Sabina Dragoi
PhD student
IBM Quantum Research & ETH Zurich

The realm of quantum algorithms - NOW

A few foundational ideas

Inspiration	Foundational Idea	Resulting Algorithms
PHYSICS	Adiabatic & Counter-Diabatic	Hamiltonian Simulation <i>Many-body physics</i>
MATH	Quantum Fourier Transform (QFT)	Shor's <i>Discrete log / factoring</i>
CS	Oracles & Machine Learning	Amplitude amplification <i>Search subroutine</i>



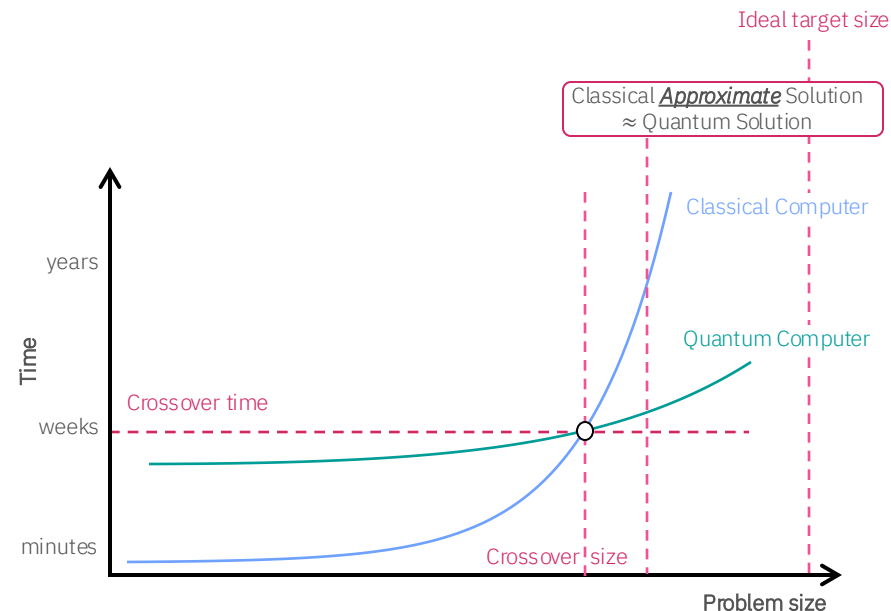
THE RIGHT PROBLEM STRUCTURE

- 1 Classically intractable

The realm of quantum algorithms - NOW

A few foundational ideas

Inspiration	Foundational Idea	Resulting Algorithms
PHYSICS	Adiabatic & Counter-Diabatic	Hamiltonian Simulation <i>Many-body physics</i>
MATH	Quantum Fourier Transform (QFT)	Shor's <i>Discrete log / factoring</i>
CS	Oracles & Machine Learning	Amplitude amplification <i>Search subroutine</i>



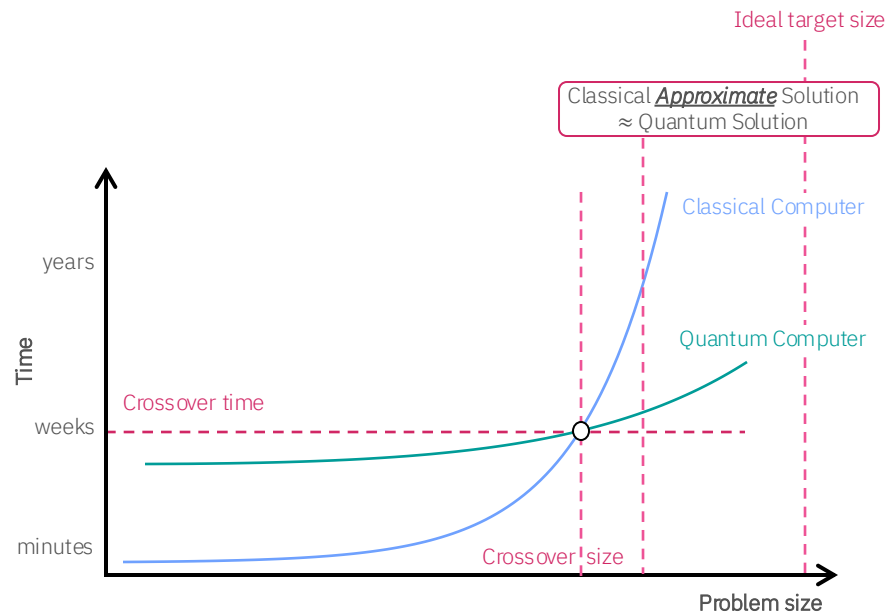
THE RIGHT PROBLEM STRUCTURE

- 1 Classically intractable
- 2 ... at problem sizes accessible on quantum hardware.

The realm of quantum algorithms - NOW

A few foundational ideas

Inspiration	Foundational Idea	Resulting Algorithms
PHYSICS	Adiabatic & Counter-Diabatic	Hamiltonian Simulation <i>Many-body physics</i>
MATH	Quantum Fourier Transform (QFT)	Shor's <i>Discrete log / factoring</i>
CS	Oracles & Machine Learning	Amplitude amplification <i>Search subroutine</i>



TWO METRICS

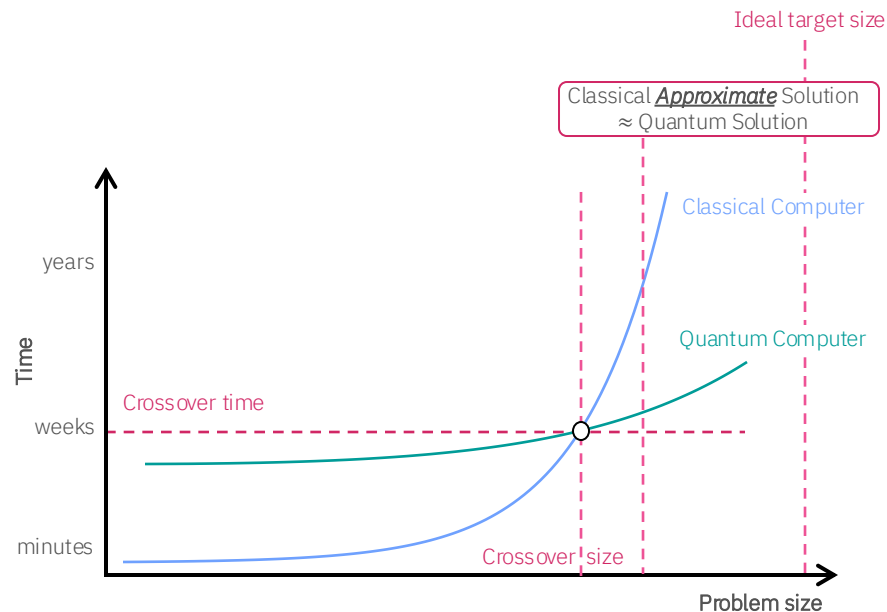
- 1 Solution quality
- 2 Resources used, e.g. Time-To-Solution

The realm of quantum algorithms - NOW

A few foundational ideas

Inspiration	Foundational Idea	Resulting Algorithms
PHYSICS	Adiabatic & Counter-Diabatic	Hamiltonian Simulation <i>Many-body physics</i>
MATH	Quantum Fourier Transform (QFT)	Shor's <i>Discrete log / factoring</i>
CS	Oracles & Machine Learning	Amplitude amplification <i>Search subroutine</i>

**NOISY
HARD-
WARE**



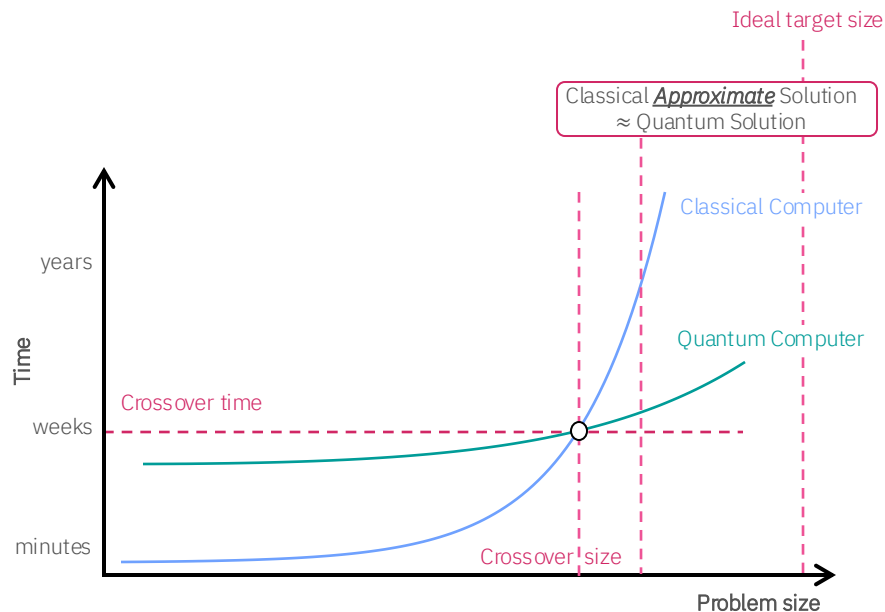
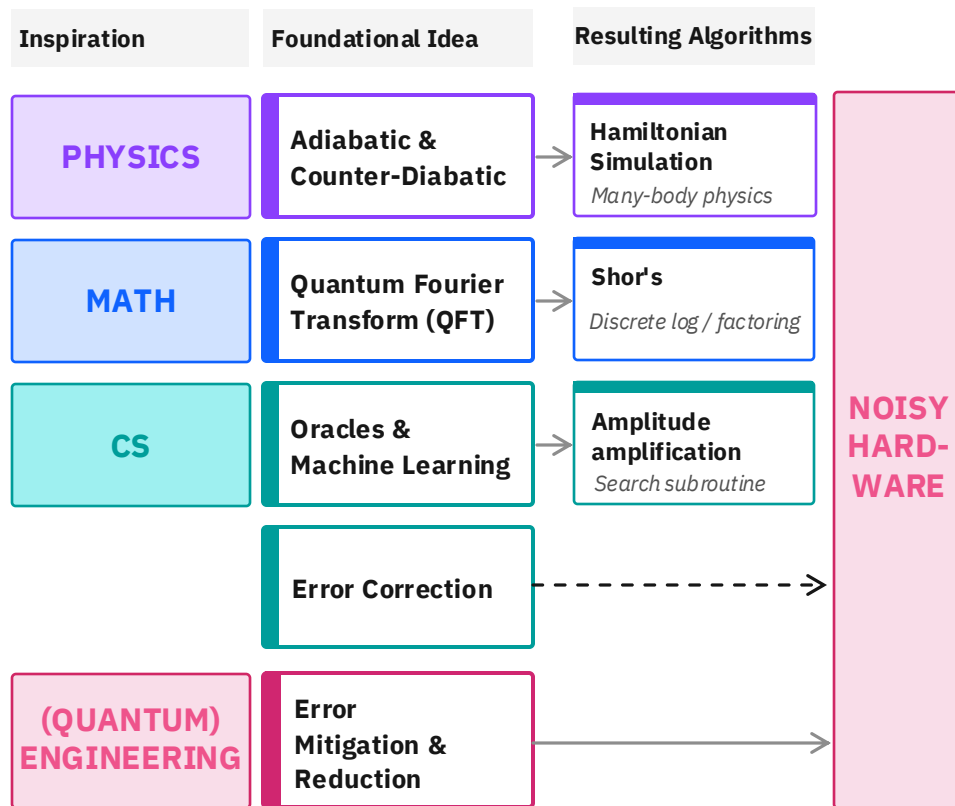
TWO METRICS

- 1 Solution quality
- 2 Resources used, e.g. Time-To-Solution

Detrimental
to both

The realm of quantum algorithms - NOW

A few foundational ideas

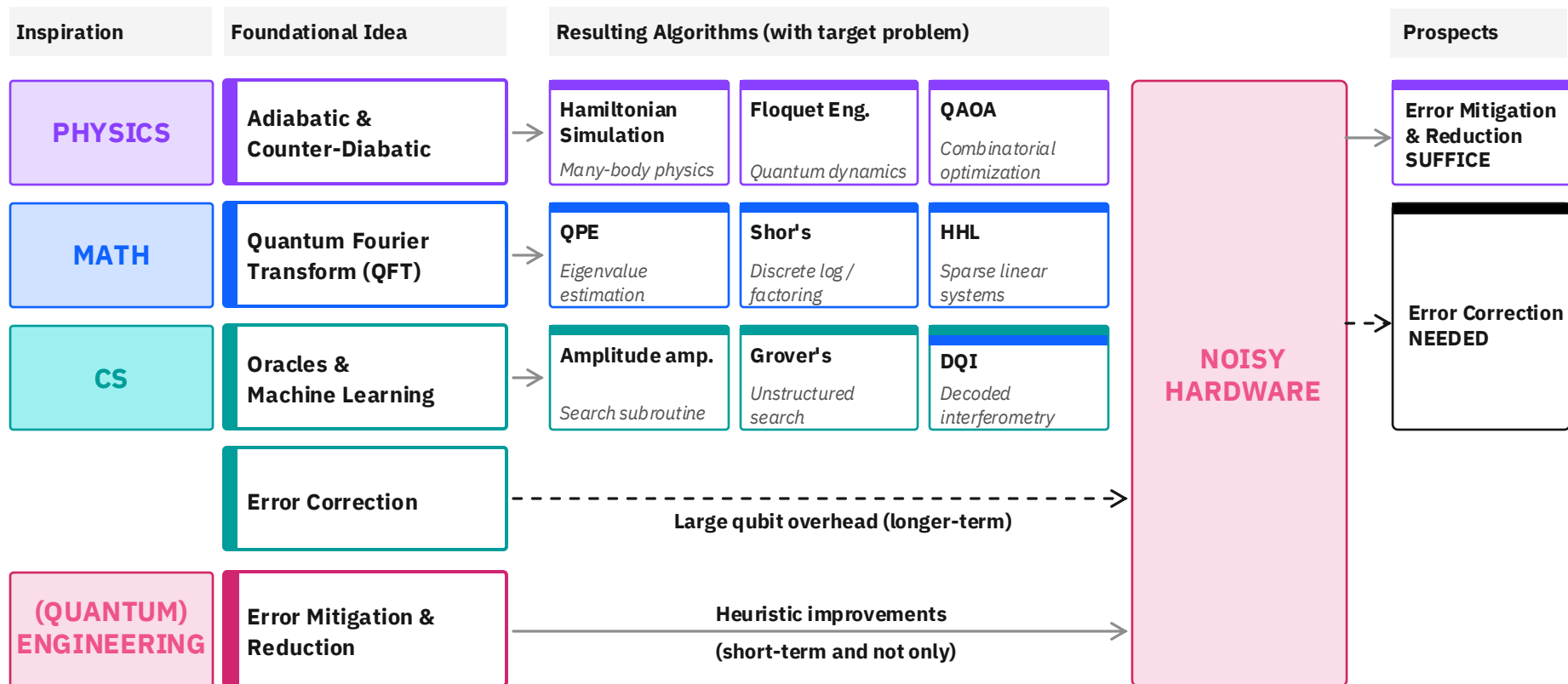


TWO METRICS

- 1 Solution quality
- 2 Resources used, e.g. Time-To-Solution

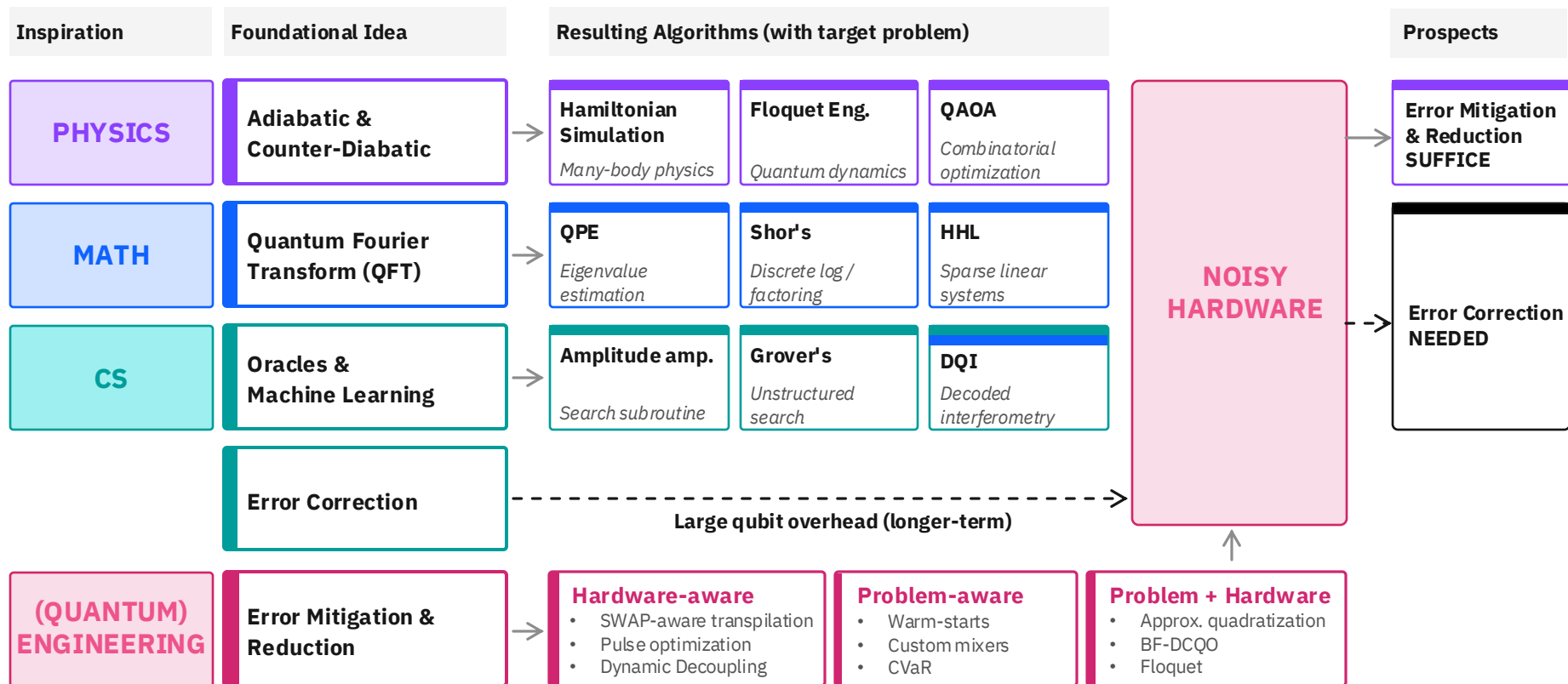
The realm of quantum algorithms - NOW

A few foundational ideas



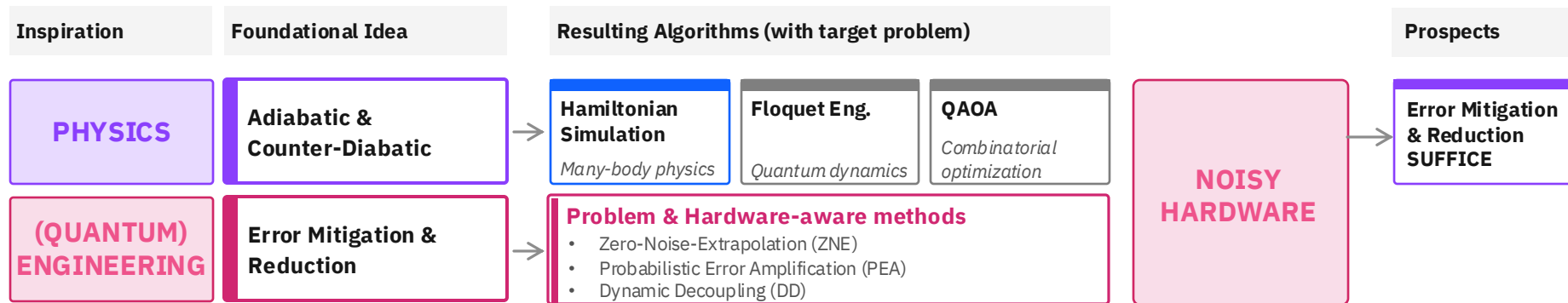
The realm of quantum algorithms - NOW

A few foundational ideas, many variants



The realm of quantum algorithms - NOW

Error mitigation is enough to achieve 'quantum utility' – in terms of solution quality



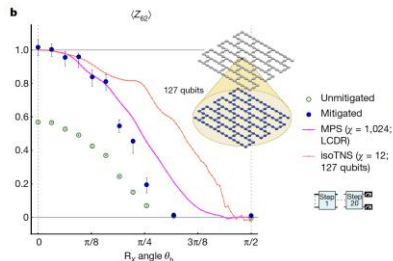
NATURE • 618, 500(2023) • cover

Evidence for the utility of quantum computing before fault tolerance

Y. Kim, A. Eddins, A. Kandala et al.

Kicked-Ising dynamics on 127-qubit Eagle — beyond brute-force classical simulation.

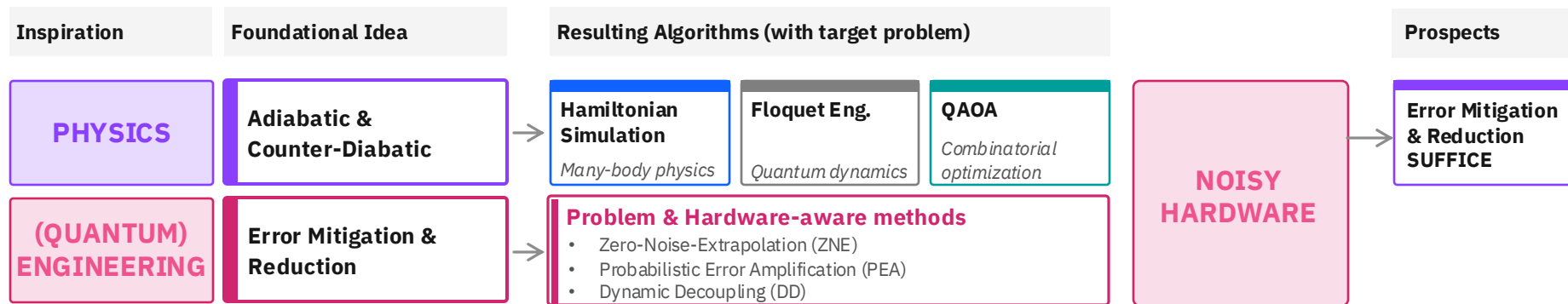
Coined the term "quantum utility".



Error mitigation: ZNE + PEA

The realm of quantum algorithms - NOW

Error mitigation is enough to achieve 'quantum utility' – in terms of *solution quality*, and *Time-To-Solution*

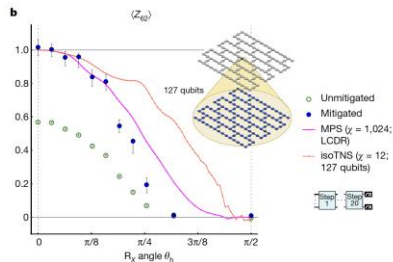


NATURE · 618, 500(2023) · cover

Evidence for the utility of quantum computing before fault tolerance

Y. Kim, A. Eddins, A. Kandala et al.

Kicked-Ising dynamics on 127-qubit Eagle — beyond brute-force classical simulation.
Coined the term "quantum utility".



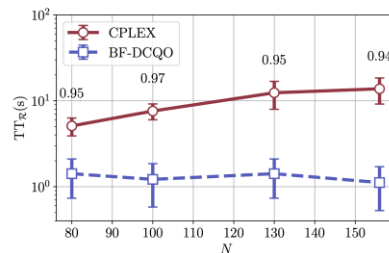
Error mitigation: ZNE + PEA

arXiv:2505.08663

Runtime Quantum Advantage with Digital Quantum Optimization

P. Chandarana et al. (Kipu,)

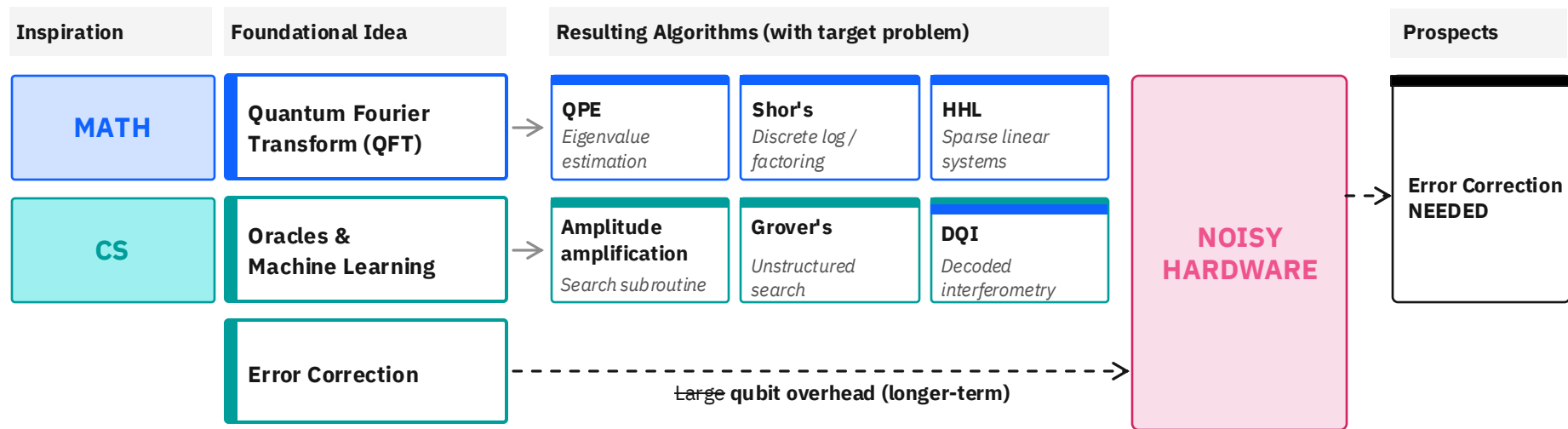
156-qubit Heron reaches desired performance *faster* than Simulated Annealing for Higher-Order Combinatorial Optimization Problem



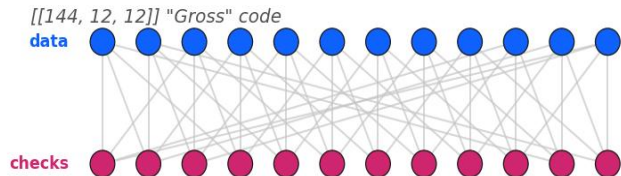
Error mitigation: CVaR, DD, post-selection based on probabilistic bit-flips

The realm of quantum algorithms - NEXT

... but Error Correction is the future



NATURE · 627, 778 (2024)



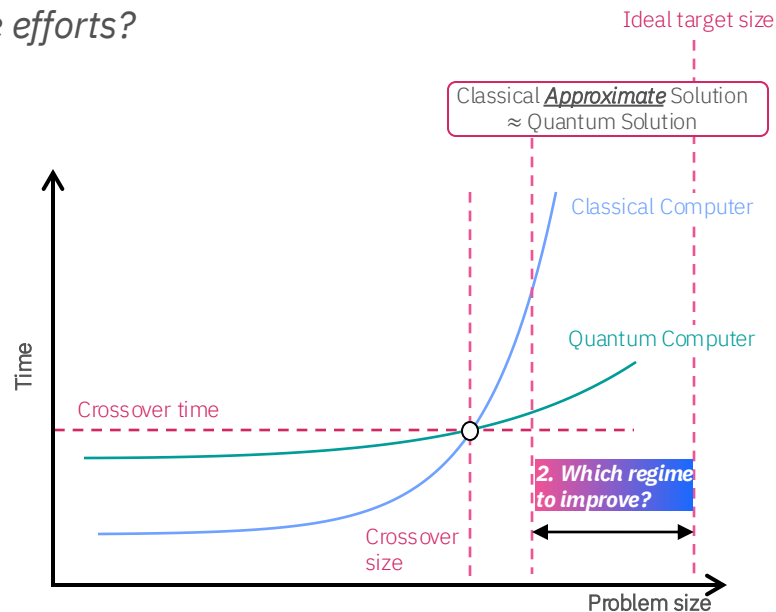
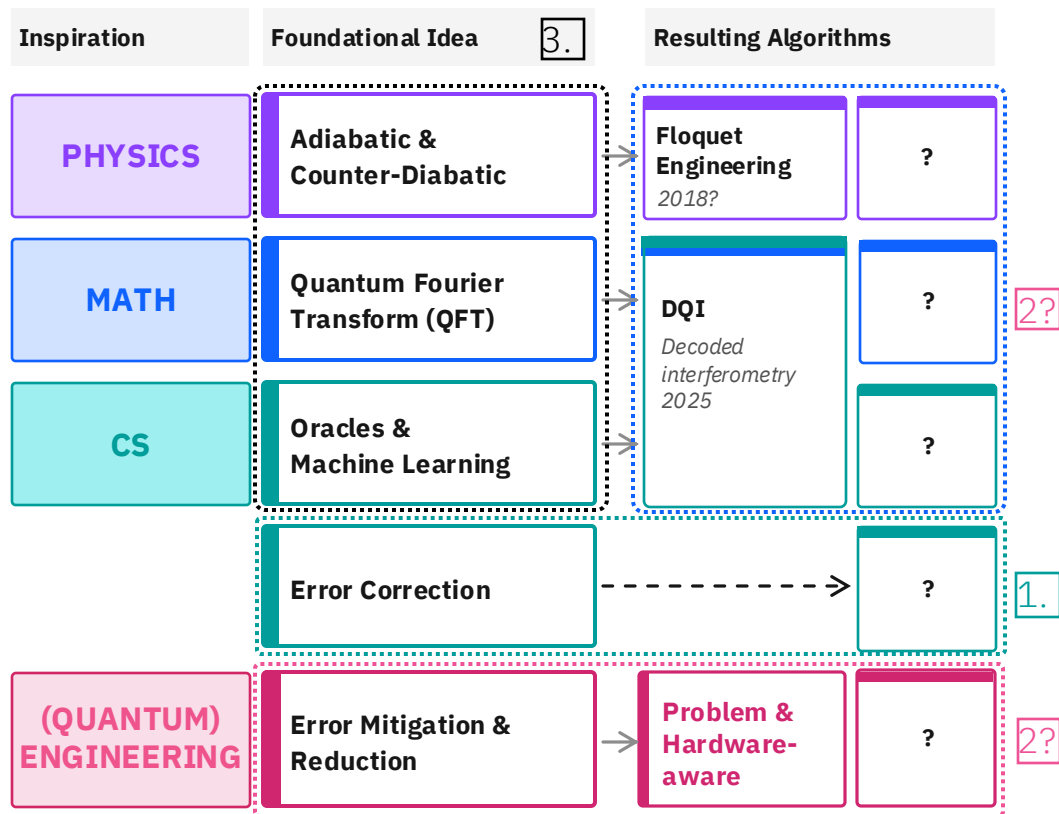
High-threshold and low-overhead fault-tolerant quantum memory

S. Bravyi, A. Cross, J. Gambetta, D. Maslov, P. Rall, T. Yoder (IBM Quantum)

- New family of qLDPC codes: bivariate bicycle (BB) codes
- Flagship "Gross code" encodes $[[144, 12, 12]]$ — 12 logical qubits in 144 physical
- $\sim 10\times$ lower qubit overhead than surface codes at the same error suppression
- Foundation of IBM's Starling fault-tolerant roadmap (200 logical qubits by 2029)

The realm of quantum algorithms - NEXT

A few foundational ideas, many variants — where to invest future efforts?



Future
(in-parallel)
Directions

1. Error Correction

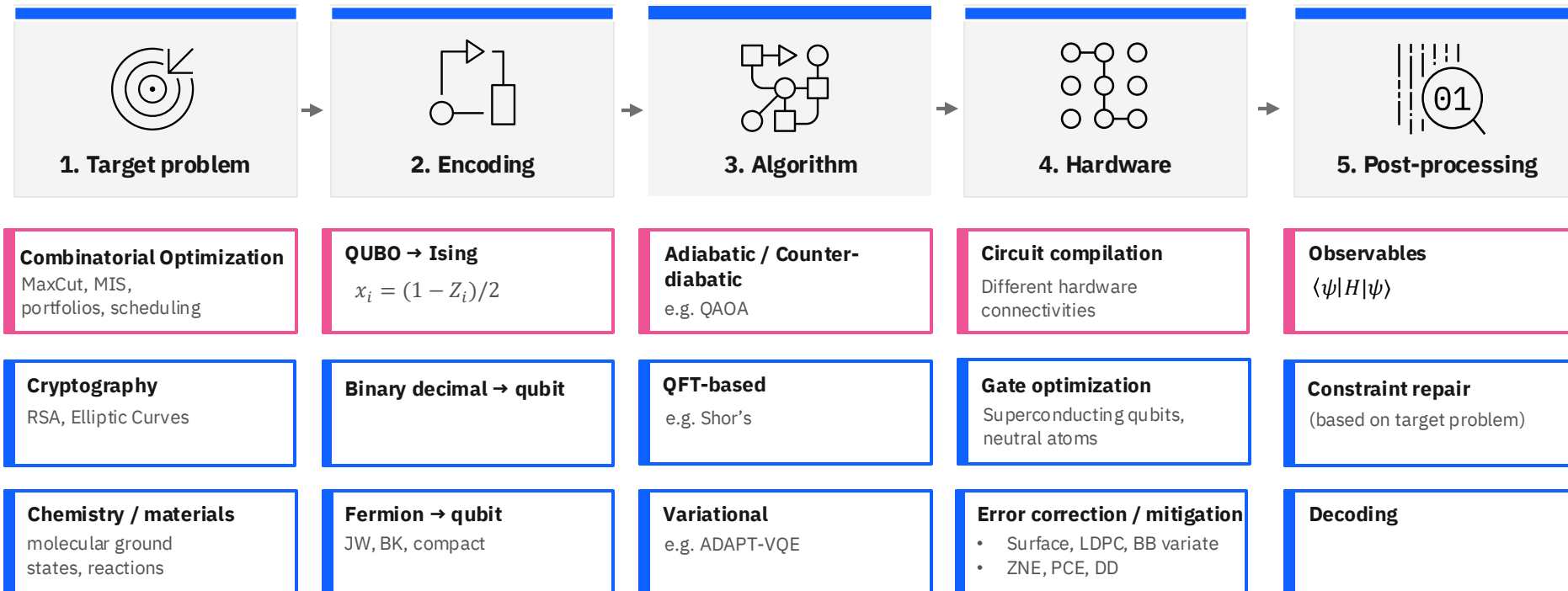
2. Error Mitigation / Reduction



New Algorithms

3. New Foundational Ideas

The full pipeline of a quantum solver



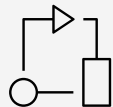
Today's plan



1. Target problem

Combinatorial Optimization

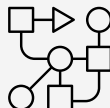
MaxCut, MIS,
portfolios, scheduling



2. Encoding

QUBO $\rightarrow$ Ising

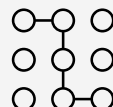
$$x_i = (1 - Z_i)/2$$



3. Algorithm

Adiabatic / Counter-diabatic evolution

e.g. QAOA



4. Hardware

Circuit compilation

Different hardware
connectivities



5. Post-processing

Observables

$$\langle \psi | H | \psi \rangle$$

From textbook ...

... to actual hardware

Problem-aware improvement:

Warm-starts

Hardware-aware improvement:

Optimized routing

Discussion
and Outlook

< \ quantum[®]

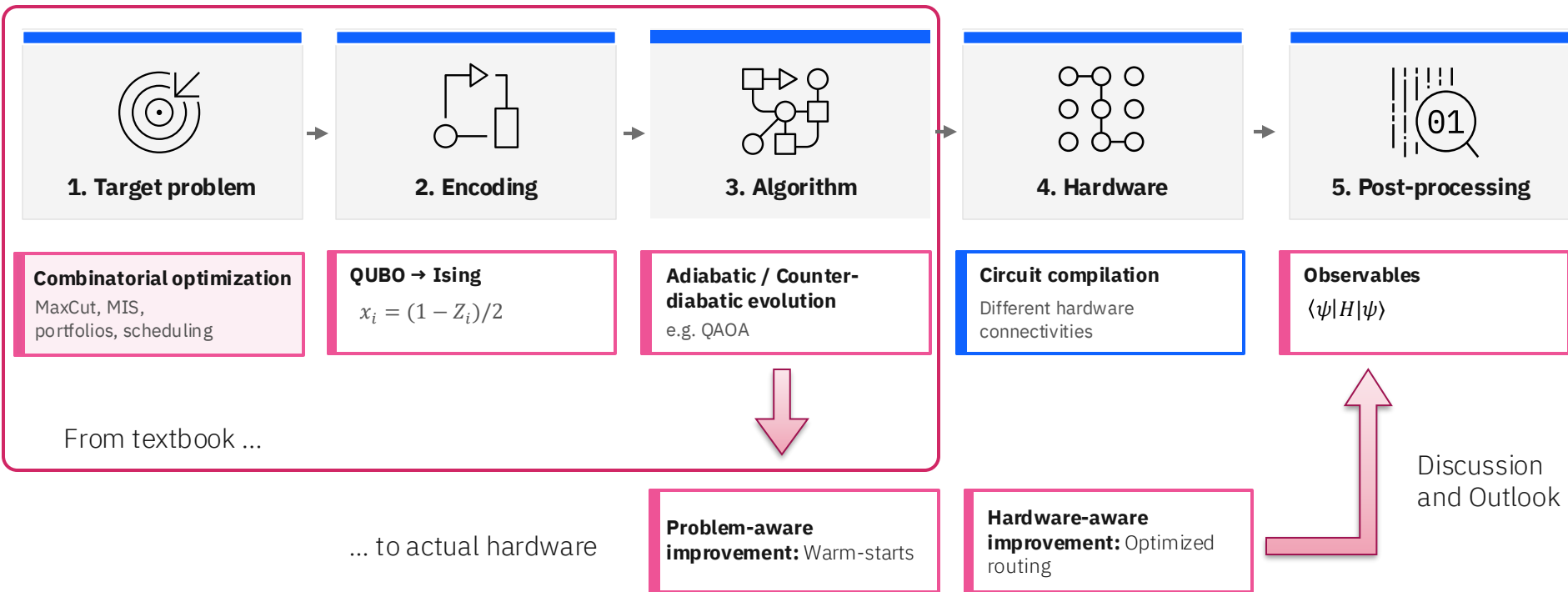
Warm-starting quantum optimization

Daniel J. Egger¹, Jakub Mareček², and Stefan Woerner¹

Scaling of the quantum approximate optimization algorithm on superconducting qubit based hardware

Johannes Weidenfeller^{1,2}, Lucia C. Valor¹, Julien Gacon^{1,3}, Caroline Tornow^{1,2}, Luciano Bello¹, Stefan Woerner¹, and Daniel J. Egger¹

Today's plan



Optimization

Continuous variables

Integer variables and mixed continuous/integer

Classical

Methods

Gradients

→ *very efficient classical solvers*

Methods

- Brute force (DFS, BFS)
- Branch-and-Bound
- Warm-start
- Relaxations
 - $\mathbf{x}_i \in \{0, 1\} \rightarrow \mathbf{x}_i \in [0, 1]$
 - Semi-definite programming

→ *(can be) NP-hard/complete*

Packages

- CPLEX
- Gurobi

Quantum

Noisy hardware

Fault tolerant

Methods (mostly heuristic)

- **Adiabatic algorithms**

[Farhi (2021), Boixo (2009)]

- **Approximate optimization (QAOA)**

[Farhi (2014), Bravyi (2020), Egger (2021), etc.]

- **Quantum methods for simulated annealing**

[Szegedy (2004), Somma (2008), Lemieux (2020)]

- **Counterdiabatic algorithms**

[Hegade (2022), Cadavid (2024), Romero (2025), Zhang (2024)]

- **Bias-field counterdiabatic**

[Cadavid (2024), Romero (2025), Chandarana (2025)]



Example: Portfolio optimization

Goal: Select the best combination of n assets to **maximize return** and **minimize risk**.

Continuous variables

Integer variables and mixed continuous/integer

Classical

Quantum

$$\min_{w \in \mathbb{R}^n} w^T \Sigma w - q \mu^T w$$

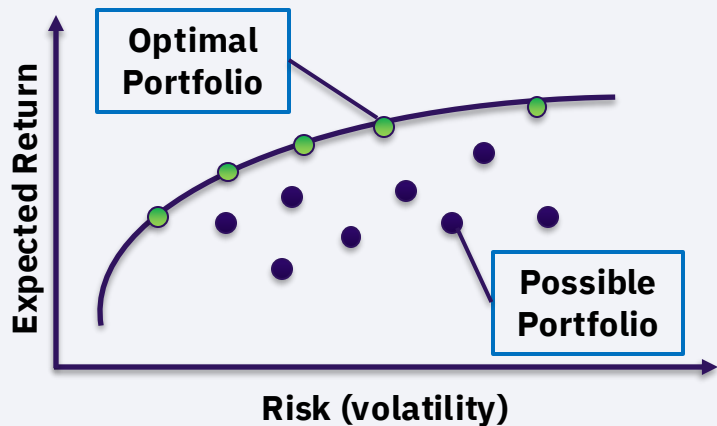
such that $\sum w_i = 1$

Mixed formulation: e.g.,
continuous weights but
constraints introduce
binary variables.

Discrete formulation

$$\min_{x \in \{0,1\}^n} x^T \Sigma x - q \mu^T x$$

such that $\sum x_i = B$



Example: Portfolio optimization

Goal: Select the best combination of n assets to **maximize return** and **minimize risk**.

Continuous variables

Integer variables and mixed continuous/integer

Classical

$$\min_{w \in \mathbb{R}^n} w^T \Sigma w - q \mu^T w$$

such that $\sum w_i = 1$

Mixed formulation: e.g., continuous weights but constraints introduce binary variables.

Discrete formulation

$$\min_{x \in \{0,1\}^n} x^T \Sigma x - q \mu^T x$$

such that $\sum x_i = B$

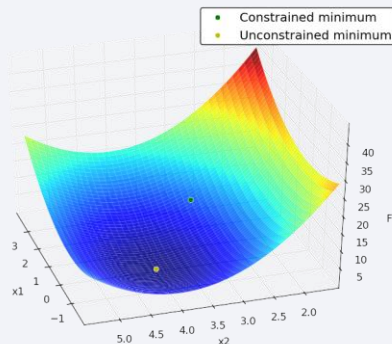
Quadratic programs

Minimize $x^T Q_0 x + c^T x$

Subject to $Ax \leq b$

$$x^T Q_i x + a_i^T x \leq r_i \text{ for } 1, \dots, i, \dots, q$$

$$l_i \leq x_i \leq u_i \text{ for } 1, \dots, i, \dots, n$$



Classical Optimization Framework

Continuous variables

Integer variables and mixed continuous/integer

Quadratic programs

Minimize $x^T Q_0 x + c^T x$

Subject to $Ax \leq b$

$x^T Q_i x + a_i^T x \leq r_i$ for $1, \dots, i, \dots, q$

$l_i \leq x_i \leq u_i$ for $1, \dots, i, \dots, n$

Easy transform

Quadratic Binary Optimization

Minimize $x^T Q_0 x + c^T x$
 $x \in \{0,1\}^n$

Integer to binary conversion

Integer variables are
converted to binary ones

$$0 \leq x \leq 7$$

Replaced by three binary
variables

$$x_0, x_1, x_2 \in \{0, 1\}$$

Classical Optimization Framework

Continuous variables

Integer variables and mixed continuous/integer

Quadratic programs

Minimize $x^T Q_0 x + c^T x$

Subject to $Ax \leq b$

$x^T Q_i x + a_i^T x \leq r_i$ for $1, \dots, i, \dots, q$

$l_i \leq x_i \leq u_i$ for $1, \dots, i, \dots, n$

Easy transform

Quadratic *Unconstrained* Binary Optimization (QUBO)

Minimize $x^T Q_0 x + c^T x$
 $x \in \{0,1\}^n$

Integer to binary conversion

Integer variables are converted to binary ones
 $0 \leq x \leq 7$

Replaced by three binary variables

$x_0, x_1, x_2 \in \{0, 1\}$

Inequality constraints

The constraint
 $Ax \leq b$

Is recast to
 $Ax - b + s = 0$

& slack variable $s \geq 0$

Equality constraints

The constraint
 $Ax = b$

is recast to
 $\min_{x \in \{0,1\}^n} M(Ax - b)^2$

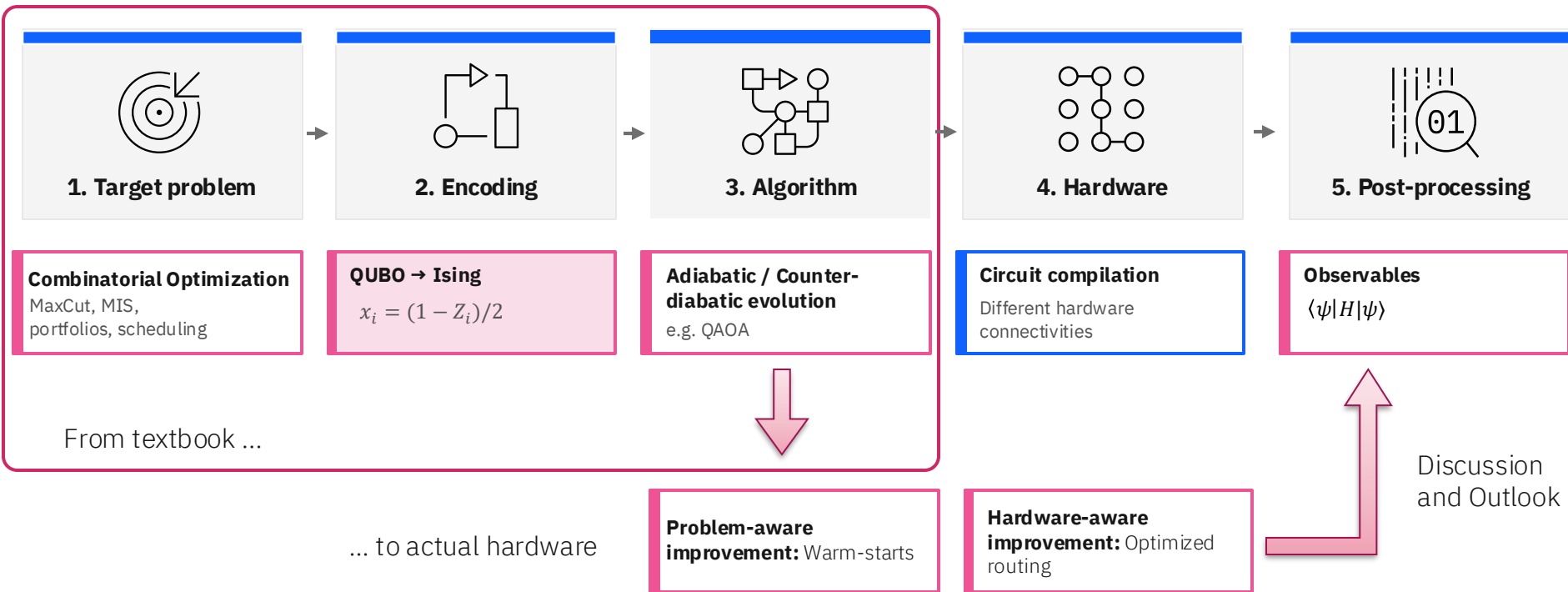
- Combinatorial optimization deals with problems like

$$\min_{\mathbf{x} \in \{0,1\}^n} f(\mathbf{x}),$$

but more complex problem classes exists.

- Describes many **NP-Complete** problems (e.g. Portfolio Optimization, MaxCut)
- We will focus on **quadratic problems** where $f(\mathbf{x}) = \mathbf{x}^T \mathbf{Q} \mathbf{x}$.
- Equality and inequality constraints can be included in a **QUBO** formulation.

Today's plan



Example: MaxCut

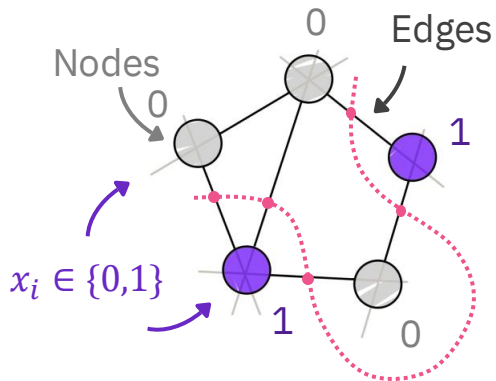
Goal: Partition a graph in two sets of nodes such that the number of edges traversed by the cut is maximum.

~~Continuous variables~~

Integer variables and mixed continuous/integer

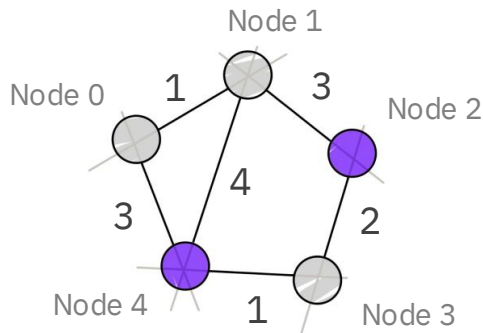
Classical

Quantum



Example: MaxCut

Goal: Partition a graph in two sets of nodes such that the number of edges traversed by the cut is maximum.



We can also assign weights to the edges, which will give us our QUBO matrix, e.g.

$$w_{ij} = \begin{pmatrix} 0 & 1 & 0 & 0 & 3 \\ 1 & 0 & 3 & 0 & 4 \\ 0 & 3 & 0 & 2 & 0 \\ 0 & 0 & 2 & 0 & 1 \\ 3 & 4 & 0 & 1 & 0 \end{pmatrix}$$

Integer variables and mixed continuous/integer

Classical

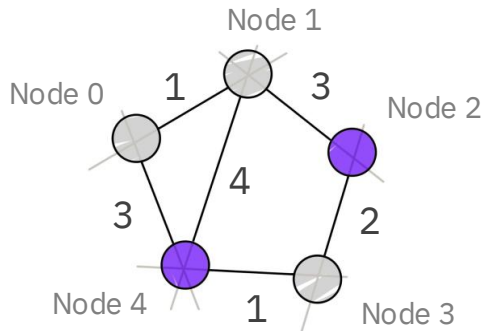
Quantum

Only edges between nodes of *different* groups contribute

$$\begin{aligned} C(x) &= \sum_{i,j=1}^n w_{ij} x_i (1 - x_j) \\ &= \sum_{i,j=1}^n w_{ij} x_i - \sum_{i,j=1}^n x_i w_{ij} x_j \end{aligned}$$

Example: MaxCut

Goal: Partition a graph in two sets of nodes such that the number of edges traversed by the cut is maximum.



$$w_{ij} = \begin{pmatrix} 0 & 1 & 0 & 0 & 3 \\ 1 & 0 & 3 & 0 & 4 \\ 0 & 3 & 0 & 2 & 0 \\ 0 & 0 & 2 & 0 & 1 \\ 3 & 4 & 0 & 1 & 0 \end{pmatrix}$$

Integer variables and mixed continuous/integer

Classical

Only edges between nodes of *different* groups contribute

$$\begin{aligned} C(x) &= \sum_{i,j=1}^n w_{ij} x_i (1 - x_j) \\ &= \sum_{i,j=1}^n w_{ij} x_i - \sum_{i,j=1}^n x_i w_{ij} x_j \end{aligned}$$

Can formulate as QUBO that minimizes $C(x)$:

$$C(x) = x^T Q x + \mu^T x$$

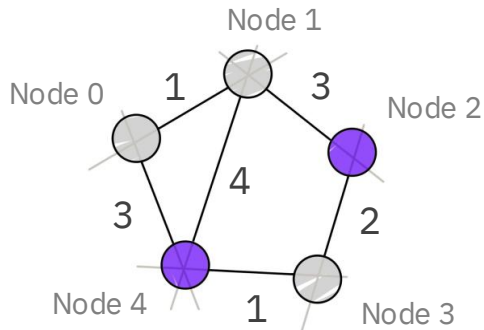
where we identify

$$Q_{ij} = -w_{ij}$$

$$\mu_i = \sum_{j=1}^n w_{ij} x_i$$

Example: MaxCut

Goal: Partition a graph in two sets of nodes such that the number of edges traversed by the cut is maximum.



$$w_{ij} = \begin{pmatrix} 0 & 1 & 0 & 0 & 3 \\ 1 & 0 & 3 & 0 & 4 \\ 0 & 3 & 0 & 2 & 0 \\ 0 & 0 & 2 & 0 & 1 \\ 3 & 4 & 0 & 1 & 0 \end{pmatrix}$$

Integer variables and mixed continuous/integer

Classical

Only edges between nodes of *different* groups contribute

$$\begin{aligned} C(x) &= \sum_{i,j=1}^n w_{ij} x_i (1 - x_j) \\ &= \sum_{i,j=1}^n w_{ij} x_i - \sum_{i,j=1}^n x_i w_{ij} x_j \end{aligned}$$

Can formulate as QUBO that minimizes $C(x)$:

$$C(x) = x^T Q x + \mu^T x$$

where we identify

$$Q_{ij} = -w_{ij}$$

$$\mu_i = \sum_{j=1}^n w_{ij} x_i$$

Example: MaxCut

Goal: Partition a graph in two sets of nodes such that the number of edges traversed by the cut is maximum.

Integer variables and mixed continuous/integer

Classical

Minimize

$$C(x) = x^T Q x + \mu^T x$$

where we identify

$$Q_{ij} = -w_{ij}$$

$$\mu_i = \sum_{j=1}^n w_{ij} x_j$$

Quantum

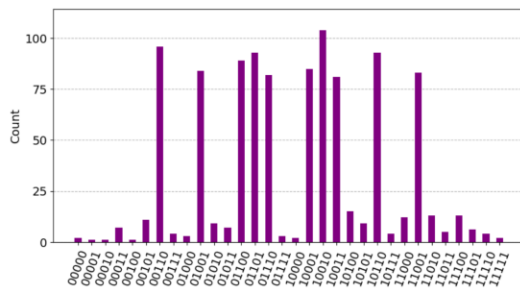
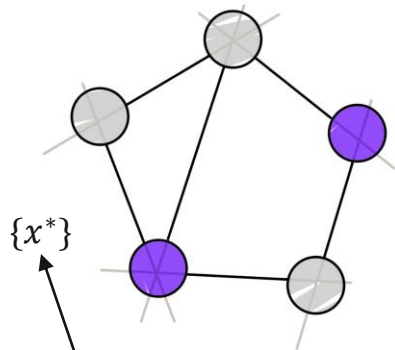
Find ground state of Hamiltonian

$$H_C |x\rangle = C(x) |x\rangle$$

which is the **lowest-energy state** of H_C , and corresponds to the **lowest-value bitstring** of C

$$|x_{\text{optimal}}\rangle = \underset{|x\rangle \in \mathcal{H}}{\operatorname{argmin}} E_{|x\rangle} = \underset{|x\rangle \in \mathcal{H}}{\operatorname{argmin}} \langle x | H | x \rangle$$

How to find H_C ?



Example: MaxCut

Goal: Partition a graph in two sets of nodes such that the number of edges traversed by the cut is maximum.

Integer variables and mixed continuous/integer

Classical

Minimize

$$C(x) = x^T Q x + \mu^T x$$

Binary values $x \in \{0,1\}$



Quantum

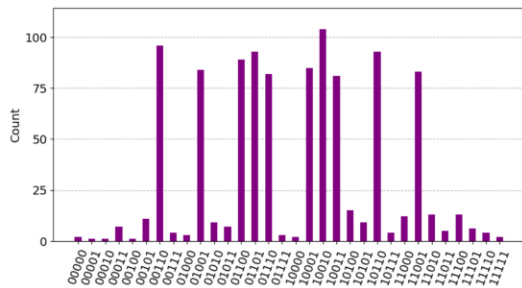
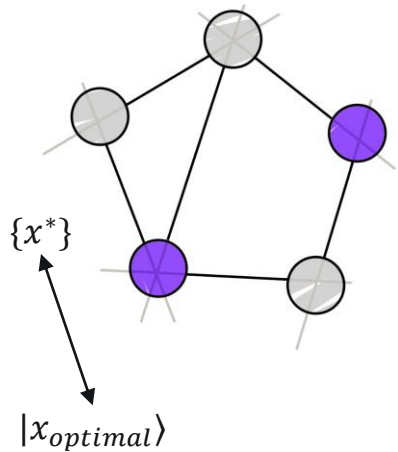
Find ground state of Hamiltonian

$$H_C|x\rangle = C(x)|x\rangle$$

How to find H_C ?

Spin eigenvalues $z_i \in \{-1,1\}$ of Pauli Z operator from $z_i = 2x_i - 1$

because $Z_i|x_i\rangle = z_i|x_i\rangle$



Mapping to a Hamiltonian

1. Map each of the optimization variables to a qubit using the substitution

$$x_i \in \{0,1\} \rightarrow x_i = \frac{1 - z_i}{2}, \quad z_i \in \{-1,1\}$$

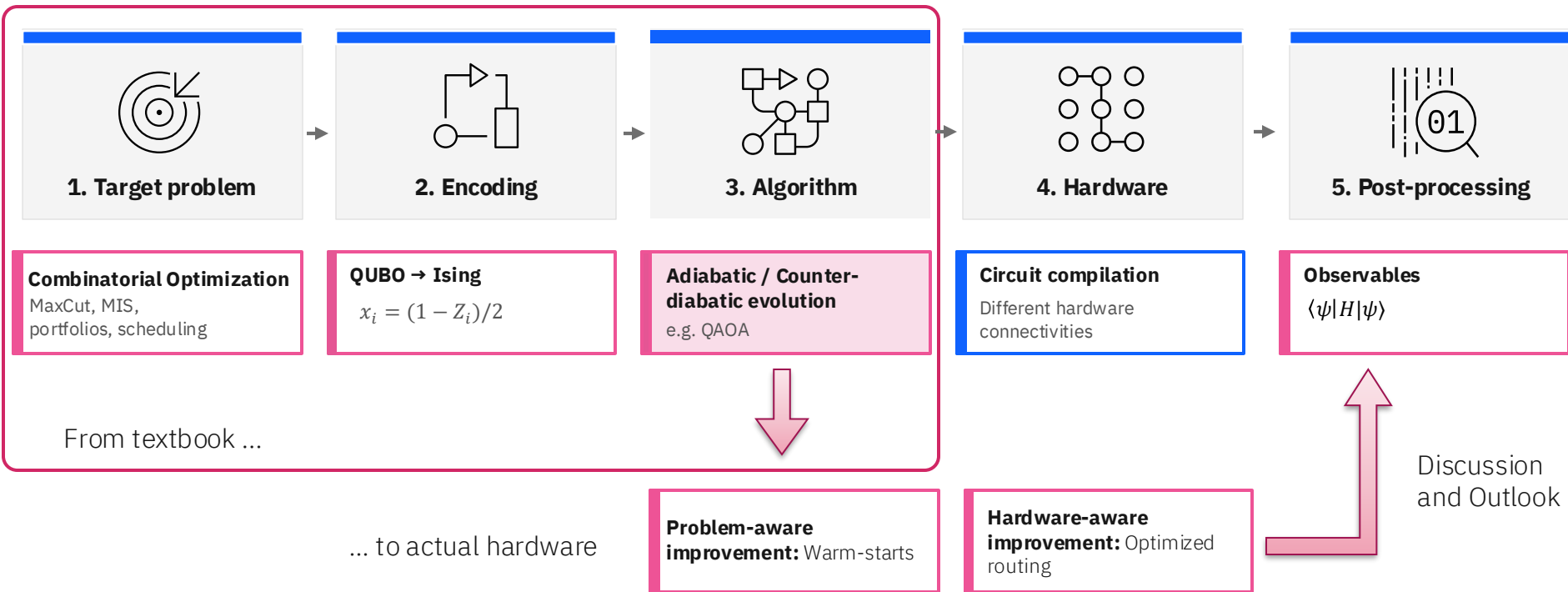
2. Promote z_i to a Pauli spin operator Z_i , where $Z_i|x_i\rangle = (-1)^{x_i}|x_i\rangle$.

3. Make the substitution in $C(x) = x^T Q x + \mu^T x$

$$\begin{aligned}
 H_C &= \sum_{ij} Q_{ij} \frac{1 - Z_i}{2} \frac{1 - Z_j}{2} + \sum_i \mu_i \frac{1 - Z_i}{2} = \sum_{ij} \frac{Q_{ij}}{4} - \sum_i \left(\sum_j \frac{Q_{ij}}{4} \right) Z_i - \overbrace{\sum_j \left(\sum_i \frac{Q_{ij}}{4} \right) Z_j}^{Q_{ij} \text{ symmetric } i \leftrightarrow j} + \sum_{ij} \frac{Q_{ij}}{4} Z_i Z_j + \sum_i \frac{\mu_i}{2} - \sum_i \frac{\mu_i}{2} Z_i = \\
 &= \underbrace{\sum_{ij} \frac{Q_{ij}}{4} Z_i Z_j - \sum_i \frac{1}{2} \left(\mu_i + \sum_j Q_{ij} \right) Z_i + \sum_{ij} \frac{Q_{ij}}{4} + \sum_i \frac{\mu_i}{2}}_{\text{This is an Ising Hamiltonian}}
 \end{aligned}$$

- In combinatorial optimization, we often encounter **QUBO** problems.
- We have seen how we can map these types of problems to **Ising Hamiltonians**, such that we can solve them on quantum computers.
- Once we have encoded our cost function in a Hamiltonian, we can minimize it by solving the **ground state problem**, i.e. finding the ground state of our system.

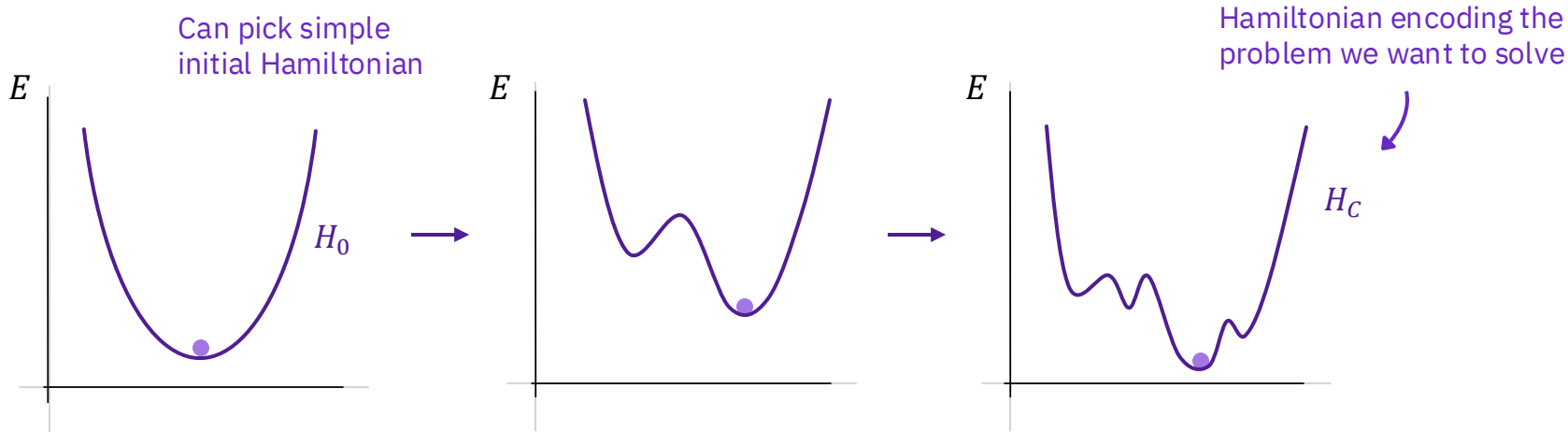
Today's plan



Adiabatic Quantum Computing

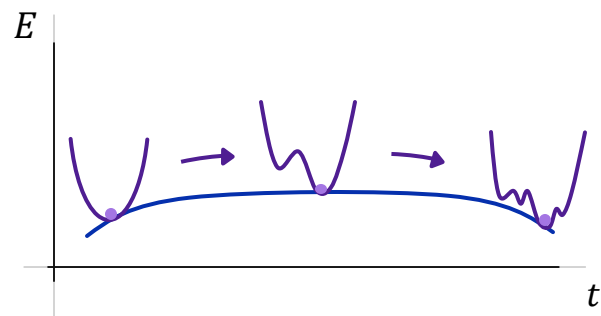
The Adiabatic Theorem

If the Hamiltonian of a system which is in its ground state is perturbed slowly enough, the system will remain in its ground state.

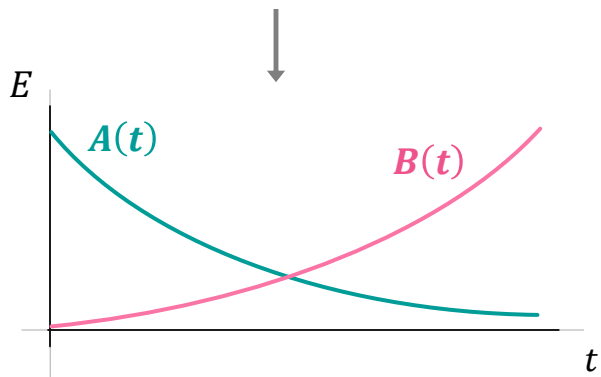


Adiabatic Quantum Computing

Using the Adiabatic Theorem to find ground states



Caveat: Tuning must be slow enough such that we remain in the ground state, per the adiabatic theorem



Tunable parameters

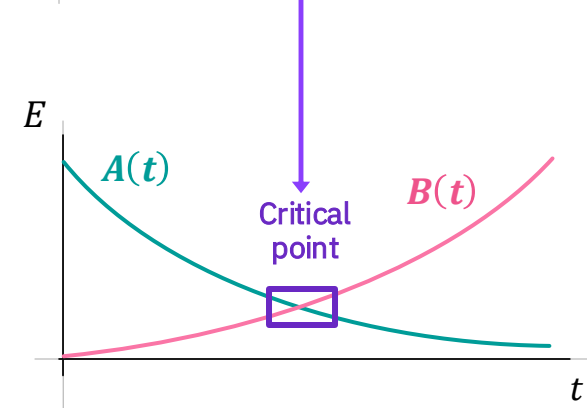
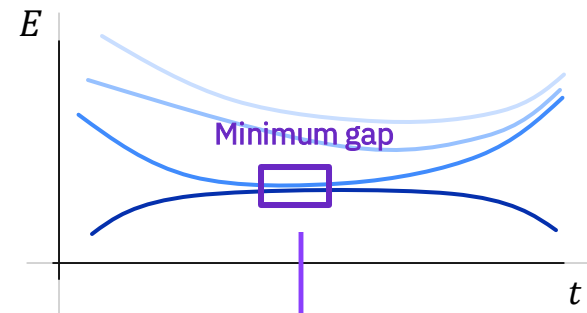
$$H(t) = \boxed{-A(t) \sum_i X_i} + \boxed{B(t) \sum_{i,j} Z_i Z_j}$$

Initial
Hamiltonian

Ground state
 $|+\rangle^{\otimes n}$
easily preparable

Problem
Hamiltonian

(may have
additional
terms)



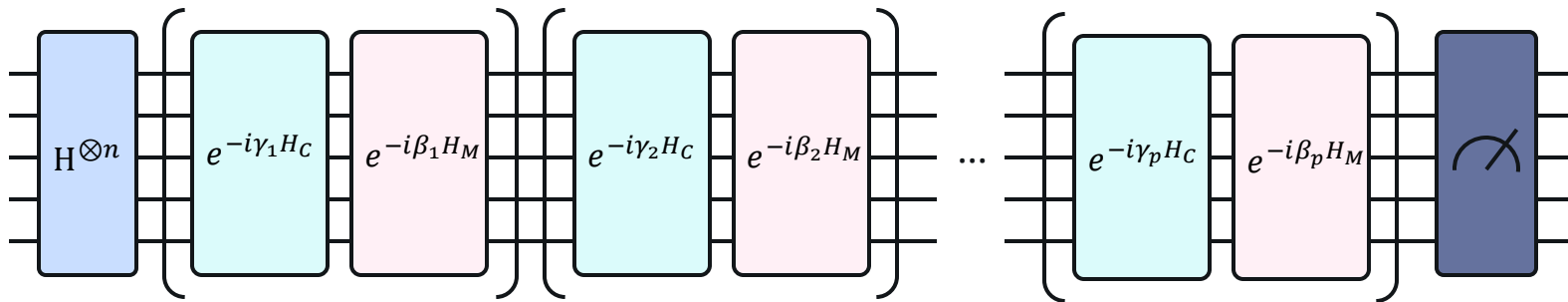
The Quantum Approximate Optimization Algorithm (QAOA): an overview

- First introduced in 2014 by Edward Farhi, Jeffrey Goldstone and Sam Gutmann
- Solves **QUBO** problems
- Is a **variational** algorithm and can be considered a special case of VQE
- Has its variational form based on the **discretization** of quantum **adiabatic annealing**

The QAOA protocol

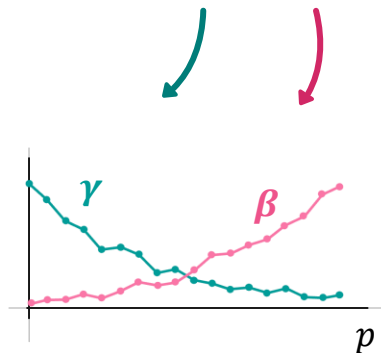
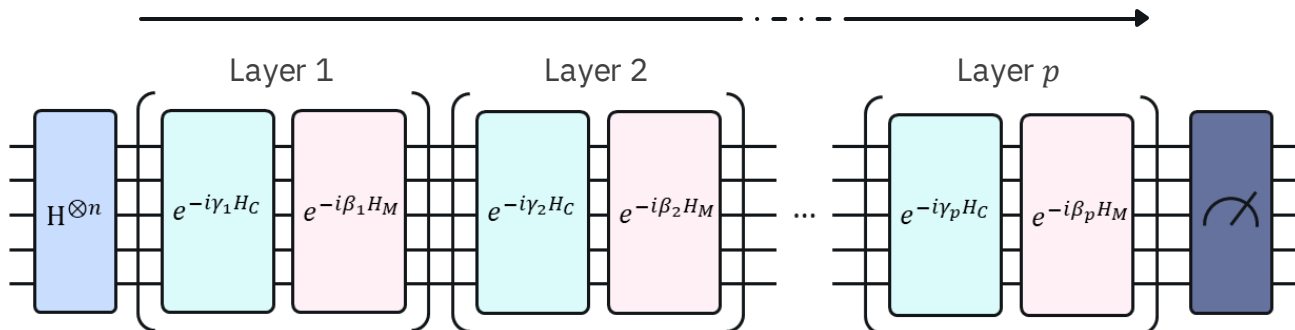
The quantum circuit

1. Define $H_M = -\sum_i X_i$ and H_C such that it encodes the cost function $f(x)$ of the optimization problem
2. Apply Hadamard gates to all qubits to begin in the ground state of H_M , i.e. $|+\rangle^{\otimes n}$
3. Apply p layers, each consisting of two terms, $e^{-i\gamma_k H_M}$ and $e^{-i\beta_k H_C}$ for $k = 1, \dots, p$
4. Measure



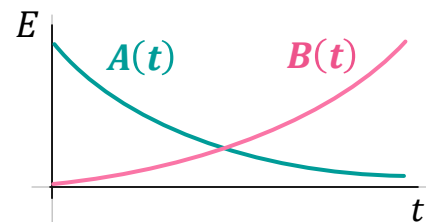
QAOA as an adiabatic schedule

As we increase p , we slow down the “annealing” and make the approximation more accurate



Parameters γ, β represent A, B in the annealing, and the depth p represents the number of “time steps”

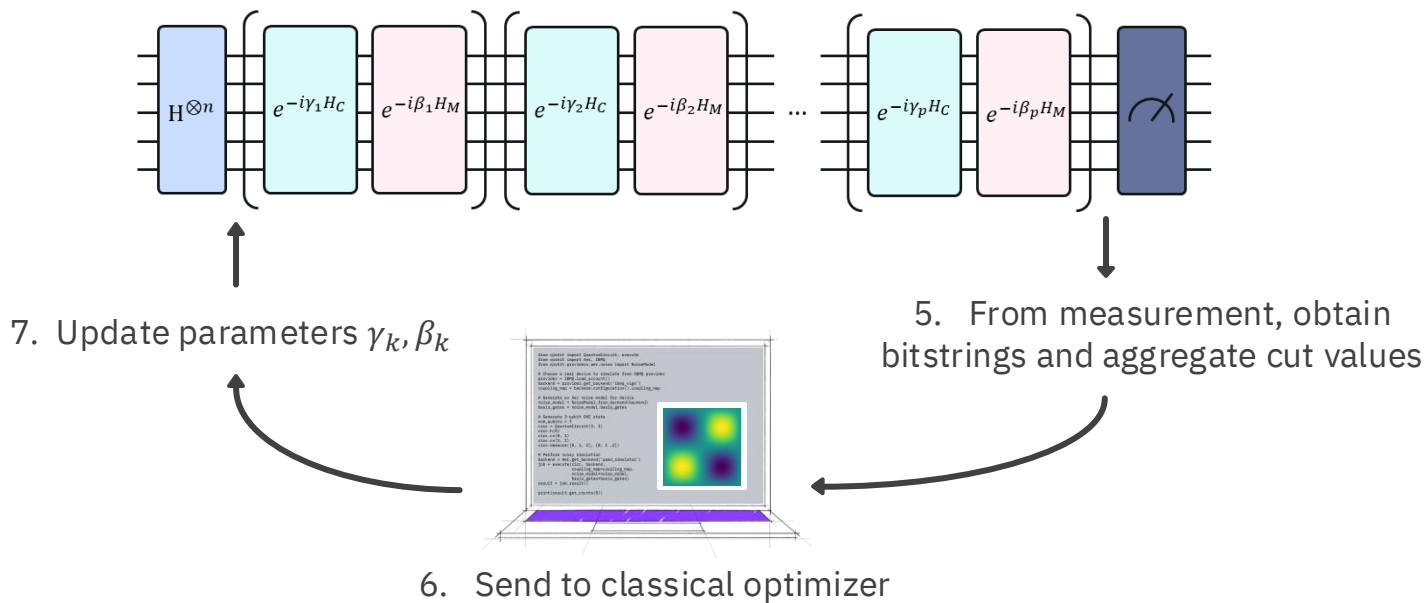
$\approx$



The QAOA protocol

The variational approach

1-4. Construct the circuit



Summary

Advantages of QAOA

- **Discretized** version of adiabatic annealing, which makes it implementable on superconducting hardware
- **Variational** version of adiabatic annealing, which makes it optimized and reduces runtimes
- Can **outperform** adiabatic annealing for subexponential runtimes
- Ability to include diabatic transitions may help overcome the **minimum gap problem**

Downsides of QAOA

- No performance **guarantees**
- **High depth** needed
- **Limited connectivity** of superconducting hardware
- Increasing number of **shots and iterations required** as problem size increases

Classical Optimization Framework

Continuous variables

Integer variables and mixed continuous/integer

Quadratic programs

Minimize $x^T Q_0 x + c^T x$

Subject to $Ax \leq b$

$x^T Q_i x + a_i^T x \leq r_i$ for $1, \dots, i, \dots, q$

$l_i \leq x_i \leq u_i$ for $1, \dots, i, \dots, n$

Easy transform

Quadratic *Unconstrained* Binary Optimization (QUBO)

Minimize $x^T Q_0 x + c^T x$
 $x \in \{0,1\}^n$

Integer to binary conversion

Integer variables are converted to binary ones
 $0 \leq x \leq 7$

Replaced by three binary variables

$x_0, x_1, x_2 \in \{0, 1\}$

Inequality constraints

The constraint
 $Ax \leq b$

Is recast to

$Ax - b + s = 0$

& slack variable $s \geq 0$

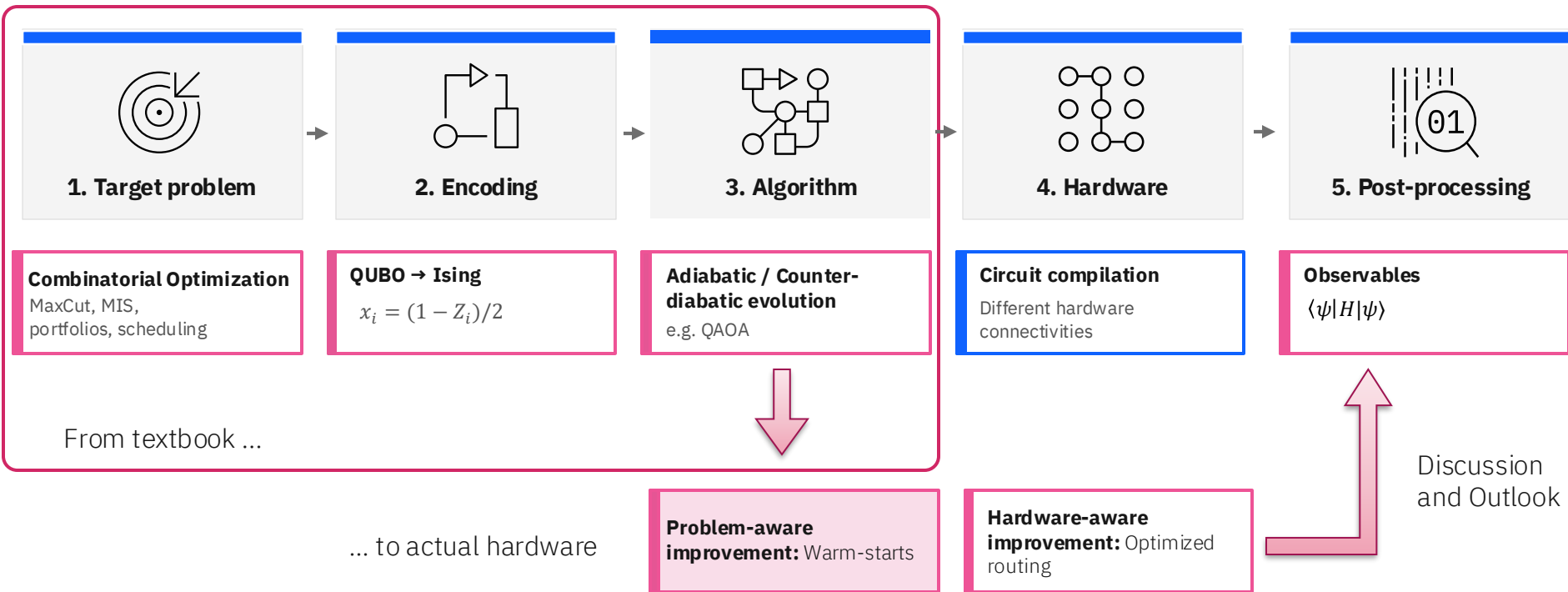
Equality constraints

The constraint
 $Ax = b$

is recast to

$\min_{x \in \{0,1\}^n} M(Ax - b)^2$

Today's plan



Warm-starting QUBOs

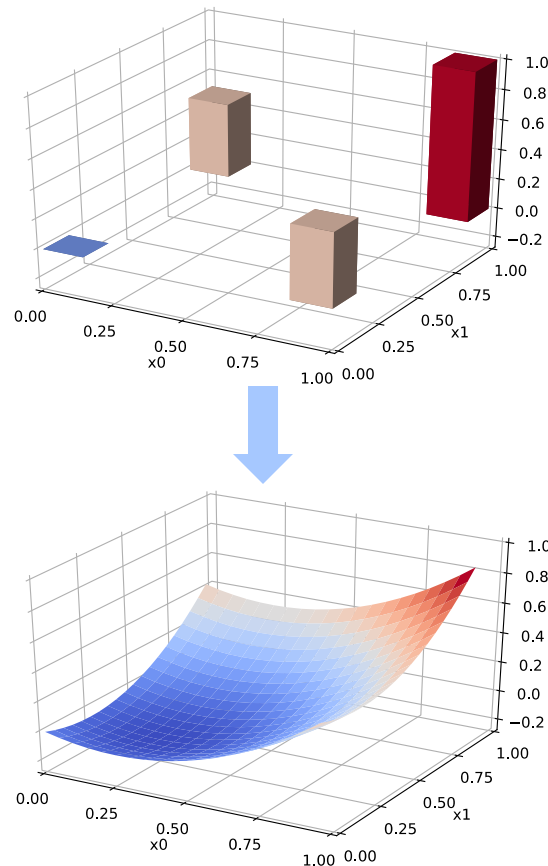
Continuous relaxation

Why 1: Convex QPs can be efficiently solved classically if Q is positive semidefinite, but QUBOs are hard to solve.

Why 2: Start in approximate solution, so we can then use local search.

How: Drop binary constraints to get Quadratic Program (QP)

$$\min_{x \in [0,1]^n} x^T Q x + b^T x$$



Warm-starting QAOA

Preparing a good initial state

Step 0: Suppose a QUBO with QP solution $\mathbf{c}^* \in [0, 1]^n$

Step 1: Prepare initial state as

$$|\psi_{\mathbf{c}^*}\rangle = \bigotimes_{i=0}^{n-1} R_Y(\theta_i)|0\rangle$$

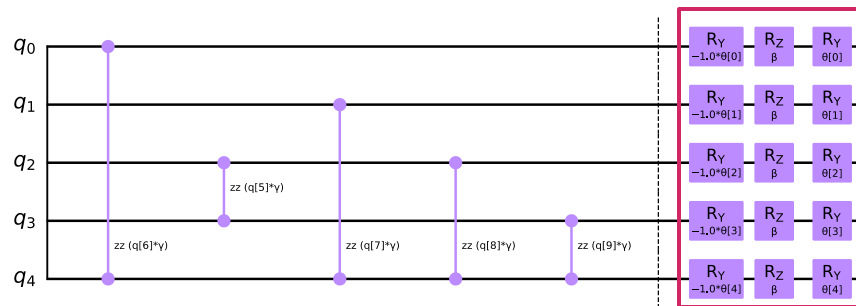
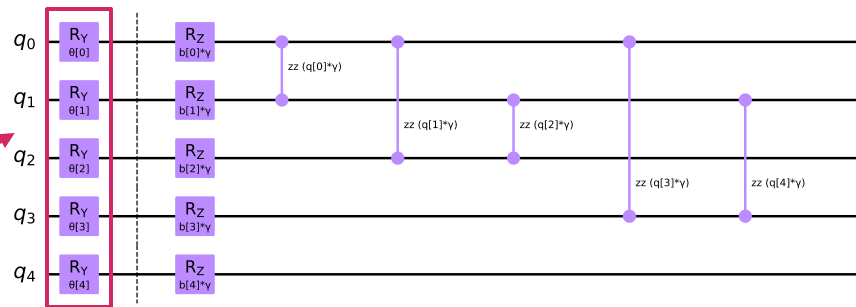
with

$$\theta_i = 2 \sin^{-1}(\sqrt{c_i^*})$$

Step 2: Use special mixing Hamiltonian $H_{X,\mathbf{c}^*} = \sum_i H_{X,\mathbf{c}^*}^i$, so that its ground state is $|\psi_{\mathbf{c}^*}\rangle$.

This works because $|\psi_{\mathbf{c}^*}\rangle$

- overlaps with optimal solution (more so that an equal superposition, as standard QAOA)
- is the ground state of $H_{X,\mathbf{c}^*}$



WS-QAOA: Portfolio optimization

Goal: minimize risk & maximize returns

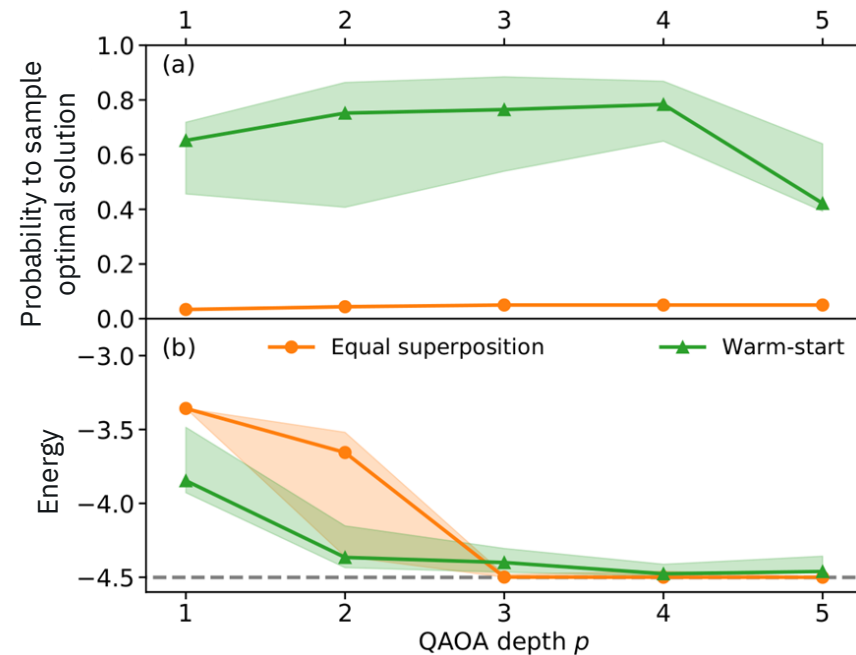
$$\max \mu^T x - q x^T \Sigma x$$

$$\text{such that } 1^T x = B$$

- Weights $x \in \{0,1\}^n$
- Returns $\mu \in \mathbb{R}^n$
- Covariance Σ $n \times n$ real matrix
- Risk-return trade-off q
- B budget

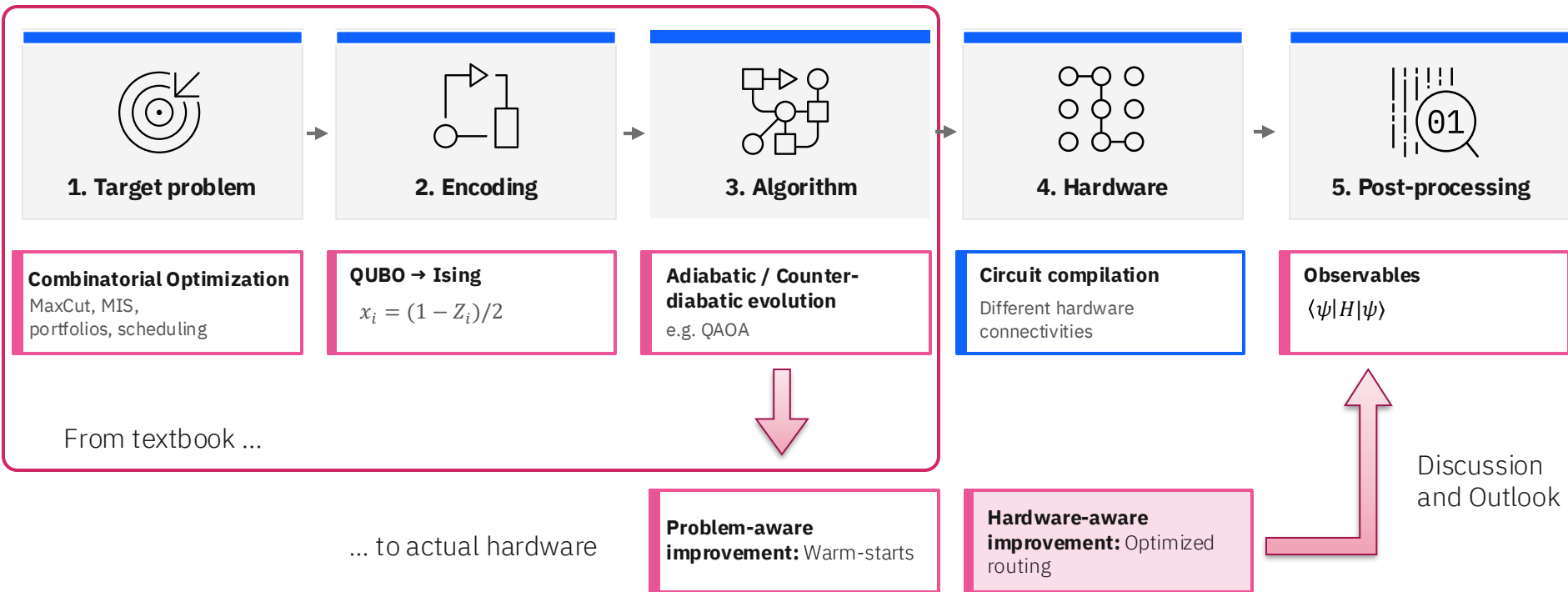
Quantum Goal: minimize energy of corresponding Ising Hamiltonian

Take-away: **WS-QAOA has better performance, especially at short depth**

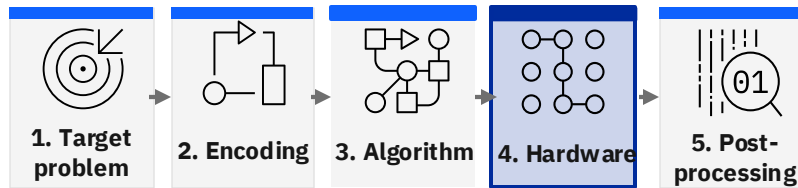


- 1 random instance with $n = 6$
- 10 repetitions for param. opt.

Today's plan



Transpilation for general circuits is *NP-hard*



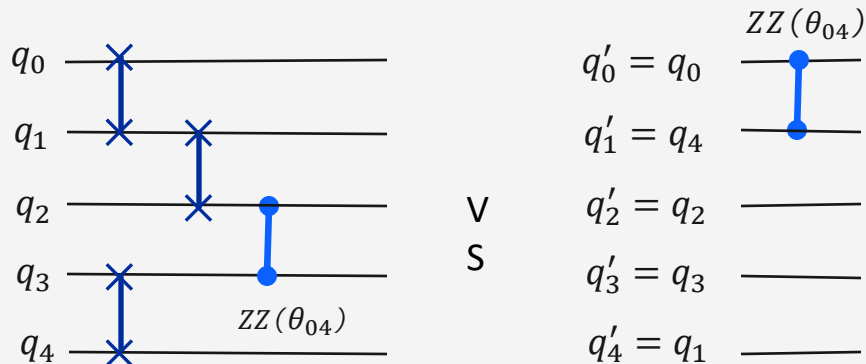
i.e. rewriting a logical circuit so it respects the hardware's coupling map.

Qubit mapping problem

Question 1: How to place qubits on hardware in the beginning π_i ?

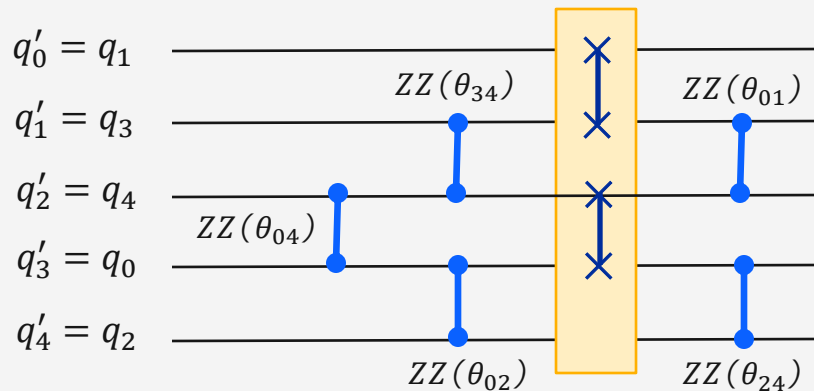
Question 2: In which order to apply gates to minimize depth, i.e. routing schedule?

$$H = Z_0 Z_4$$

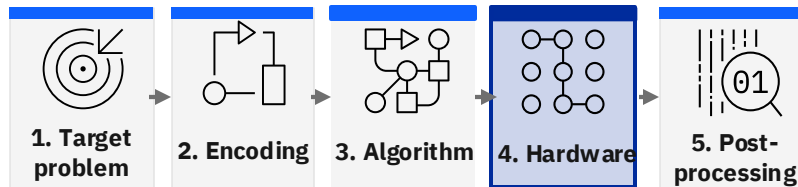


V
S

$$H = Z_0 Z_4 + Z_0 Z_2 + Z_3 Z_4 + Z_0 Z_1$$



Transpilation for general circuits is *NP-hard*



i.e. rewriting a logical circuit so it respects the hardware's coupling map.

Qubit mapping problem

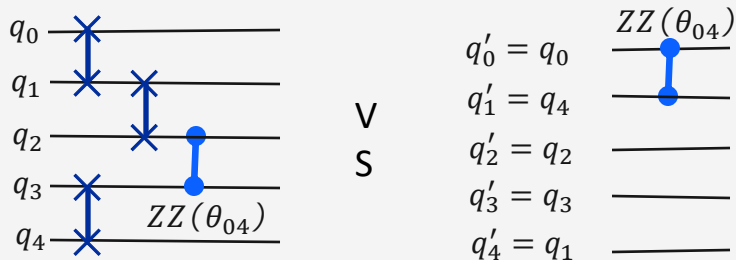
Question 1: How to place qubits on hardware in the beginning π_i ?

Question 2: In which order to apply gates to minimize depth, i.e. routing schedule?

Current (general) approach

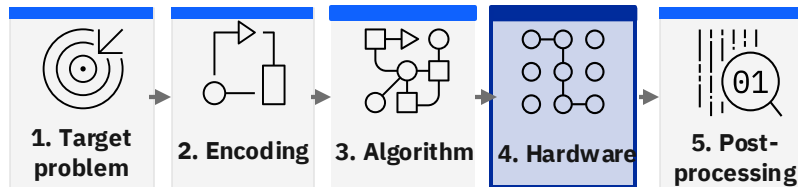
Fix random initial mapping: $\pi_i(\text{node}) = \text{qubit}$

Greedy insert SWAPs given look-ahead depth



Exponentially small chance of finding right initial map!

Transpilation for circuits with commuting gates



i.e. rewriting a logical circuit so it respects the hardware's coupling map.

Qubit mapping problem

Question 1: How to place qubits on hardware in the beginning π_i ?

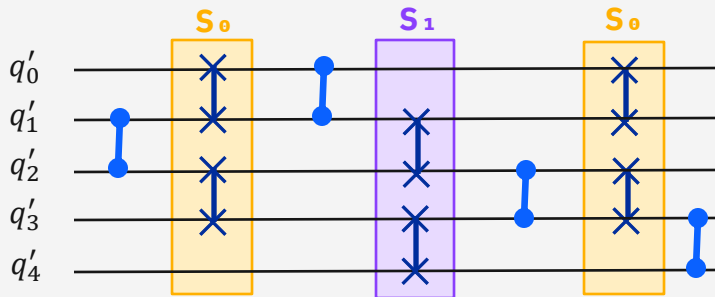
Question 2: In which order to apply gates to minimize depth, i.e. routing schedule?

SWAP strategies approach

Fix sequence of SWAP layers induces all possible interactions E'

Find almost-optimal initial mapping $\pi_i: \{q_j\} \rightarrow \{q'_j\}$

from embedding $E(\text{nodes}) \subseteq E'(\text{qubits})$

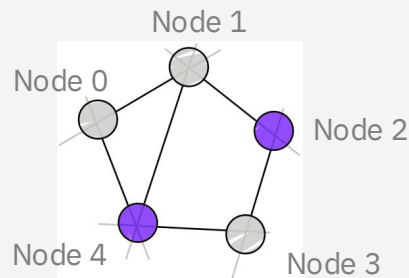


Theorem [Weidenfeller et al. 2022]:

A **fully-connected** QUBO on n qubits is **optimally** implemented in at most $n - 2$ linear SWAP layers, i.e. $\{S_0, S_1\}$.

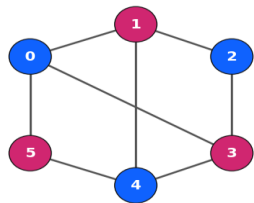
Exercise

Check that regardless of qubit placement, we need less than 3 SWAP layers to implement this graph.

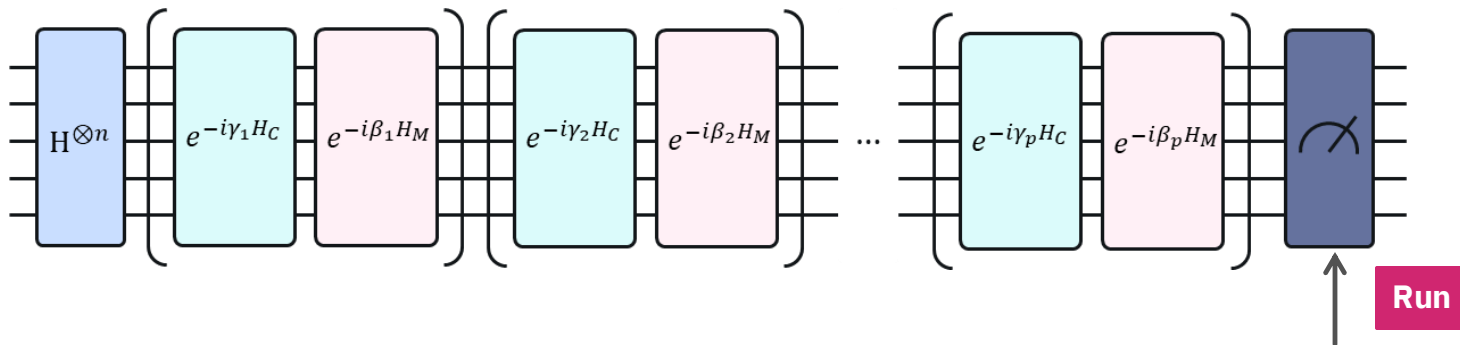


Three Qiskit tools for end-to-end QAOA

From a high-level problem statement all the way to optimized circuits ready to run on hardware



6-node graph, 3-regular



Problem modelling and encoding

qiskit-addon-opt-mapper

Get Ising cost Hamiltonian H_C from given graph $G = (V, E)$, and target problem.

qiskit.github.io/qiskit-addon-opt-mapper

Hardware-aware transpilation

qopt-best-practices

Get initial mapping $\pi_i: \{q_j\} \rightarrow \{q'_j\}$ given hardware connectivity, and SWAP strategy.

github.com/qiskit-community/qopt-best-practices

Classical Parameter training

qaoa_training_pipeline

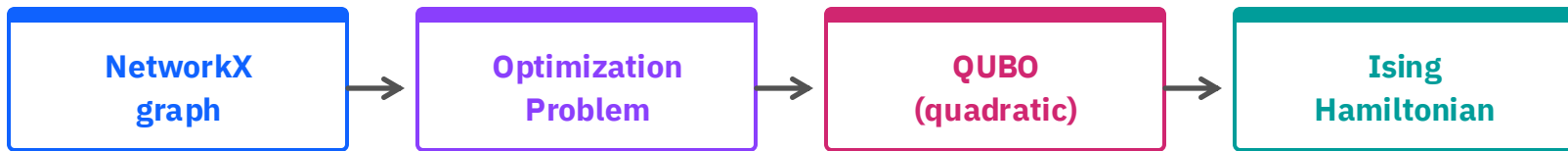
Train QAOA angles (γ, β) classically before running on hardware — quantum training is costly.

github.com/qiskit-community/qaoa_training_pipeline

Problem modeling with qiskit-addon-opt-mapper

From a NetworkX graph to a SparsePauliOp Hamiltonian in 4 lines

TOOL 1/3



```

import networkx as nx
from qiskit_addon_opt_mapper.applications import Maxcut
from qiskit_addon_opt_mapper.converters \
    import OptimizationProblemToQubo
from qiskit_addon_opt_mapper.translators import to_ising
  
```

```

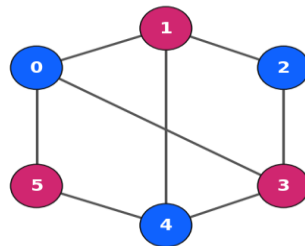
# 1. Build the problem
graph = nx.random_regular_graph(d=3, n=6, seed=42)
maxcut = Maxcut(graph)
problem = maxcut.to_optimization_problem()
  
```

```

# 2. Convert to QUBO
qubo = OptimizationProblemToQubo().convert(problem)
  
```

```

# 3. Map to Ising Hamiltonian
hamiltonian, offset = to_ising(qubo)
# SparsePauliOp(['IIIIZZ', 'IIIZZ', 'IIZZII', ...,
#               'ZIIIIZ'], coeffs=[-0.5, ...])
  
```



6-node graph,
3-regular

Built-in problems

Maxcut · TSP · Knapsack
 VehicleRouting ·
 BinPacking
 VertexCover · Clique ·
 SKModel

Also supports

- Higher-order terms (HUBO)
- Binary $\leftrightarrow$ Spin conversion
- Integer $\rightarrow$ binary encoding
- Inequality $\rightarrow$ penalty
- Validate against CPLEX, Gurobi, scipy.milp

Hardware-aware transpilation with qopt-best-practices

Three steps to a QAOA circuit that survives the noise of today's hardware

TOOL 2 / 3

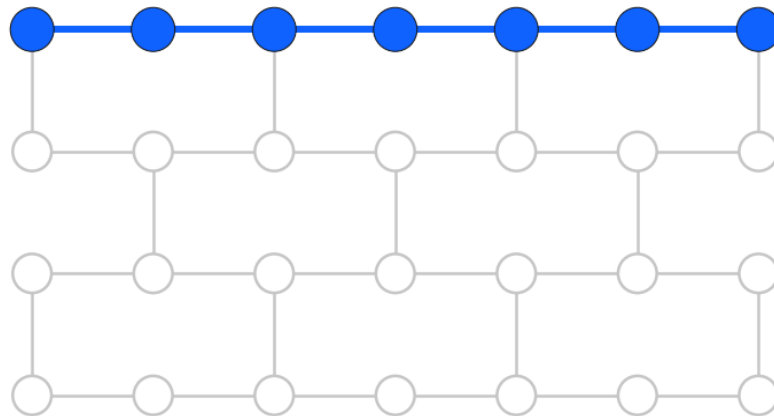
1

Find an optimal qubit subset

Match the program graph to a long path through the hardware's coupling graph.

```
lines = find_lines(  
    num_qubits=50,  
    backend=backend,  
)
```

Step 1: pick a long line through the heavy-hex



Hardware-aware transpilation with qopt-best-practices

Three steps to a QAOA circuit that survives the noise of today's hardware

TOOL 2 / 3

1

Find an optimal qubit subset

Match the program graph to a long path through the hardware's coupling graph.

```
lines = find_lines(  
    num_qubits=50,  
    backend=backend,  
)
```

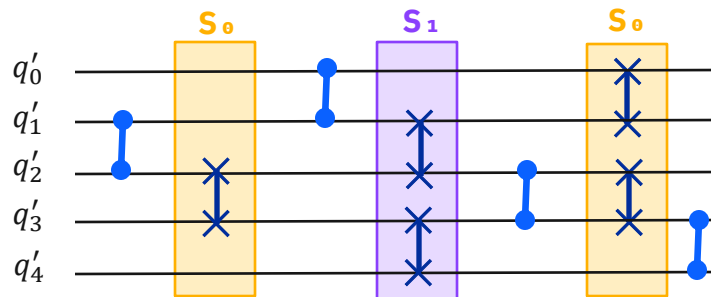
2

Apply a SWAP strategy

Exploit commutativity of QAOA cost layers: a line-based SWAP pattern needs at most $n-2$ layers to implement all two-qubit interactions.

```
swap_strategy =  
SwapStrategy.from_line(  
    line_qubits)  
  
qaoa_pm = make_qaoa_pm(  
    swap_strategy, backend)
```

Step 2: Apply SWAP strategy find all interactions available after k layers $E'(k)$



Hardware-aware transpilation with qopt-best-practices

Three steps to a QAOA circuit that survives the noise of today's hardware

TOOL 2 / 3

1

Find an optimal qubit subset

Match the program graph to a long path through the hardware's coupling graph.

```
lines = find_lines(  
    num_qubits=50,  
    backend=backend,  
)
```

2

Apply a SWAP strategy

Exploit commutativity of QAOA cost layers: a line-based SWAP pattern needs at most $n-2$ layers to implement all two-qubit interactions.

```
swap_strategy =  
    SwapStrategy.from_line(  
        line_qubits)  
  
qaoa_pm = make_qaoa_pm(  
    swap_strategy, backend)
```

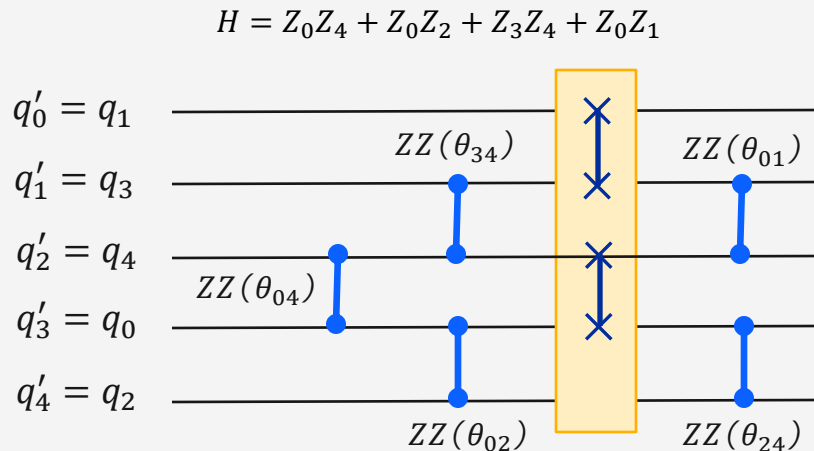
3

Find the best initial qubit map

SAT-encode "does this layout need $\leq k$ swap layers?" Binary-search k . Matsuo et al. — scales to 500+ qubits.

```
mapper =  
    SATMapper(timeout=60)  
result =  
    mapper.find_initial_mappings(  
        program_graph,  
        swap_strategy)
```

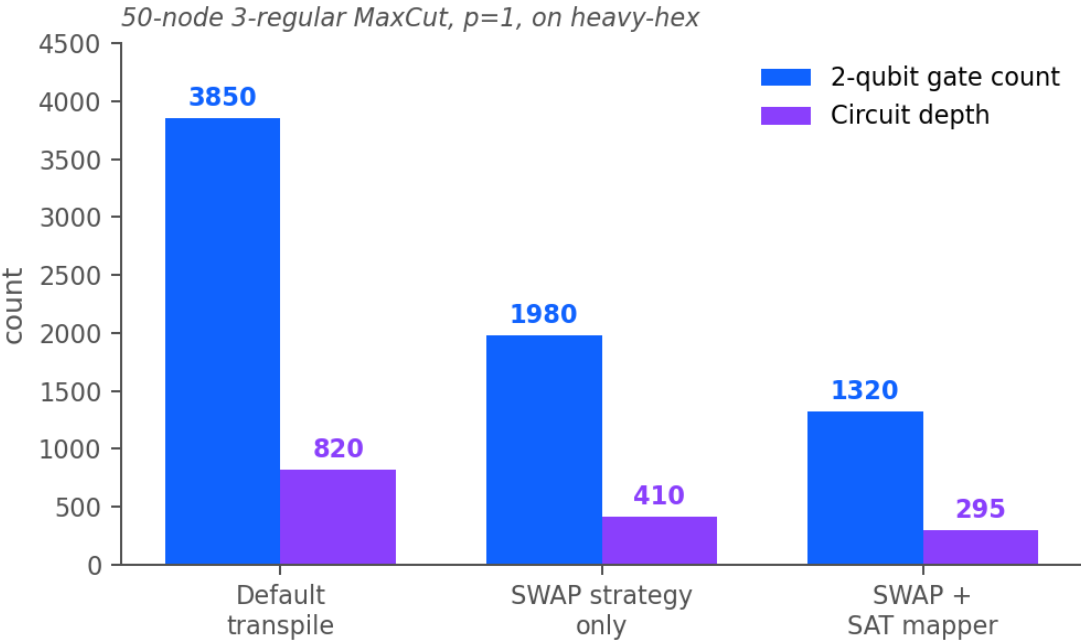
Step 3: Use SAT-solver to find the map $\pi_0 : \{q_i\} \rightarrow \{q'_i\}$ that encodes the desired interactions E in the fewest amount of SWAP layers k



Impact of each transpilation step

Each pass roughly halves the 2-qubit gate count – gates are the dominant noise source

TOOL 2 / 3



Takeaways

- Default transpilation is wasteful for circuits with only commuting gates like QAOA
- SWAP strategy: ~50% reduction
- SAT mapper: another 30%

Classical parameter training with qaoa_training_pipeline

IBM Quantum

Train QAOA angles offline — quantum training is expensive and runs into Barren Plateaus

TOOL 3 / 3

p = 1

Brute-force scan + SciPy refine

Exact 2D landscape sweep over (γ, β) .
Fast, but does not scale beyond $p=1$.



p = 2

TransitionStates from depth-1

Use depth-1 optimum to seed the search.
Light-cone evaluator gives exact energies.



p = 3

Recursion + MPS evaluator

Bond dim 64 matrix-product state.
Interpolate parameters across depths.

⚠ Beyond $p = 3$: barren plateaus dominate. For d -regular random graphs ($d = 3..11$), optima are tabulated up to $p = 3$.

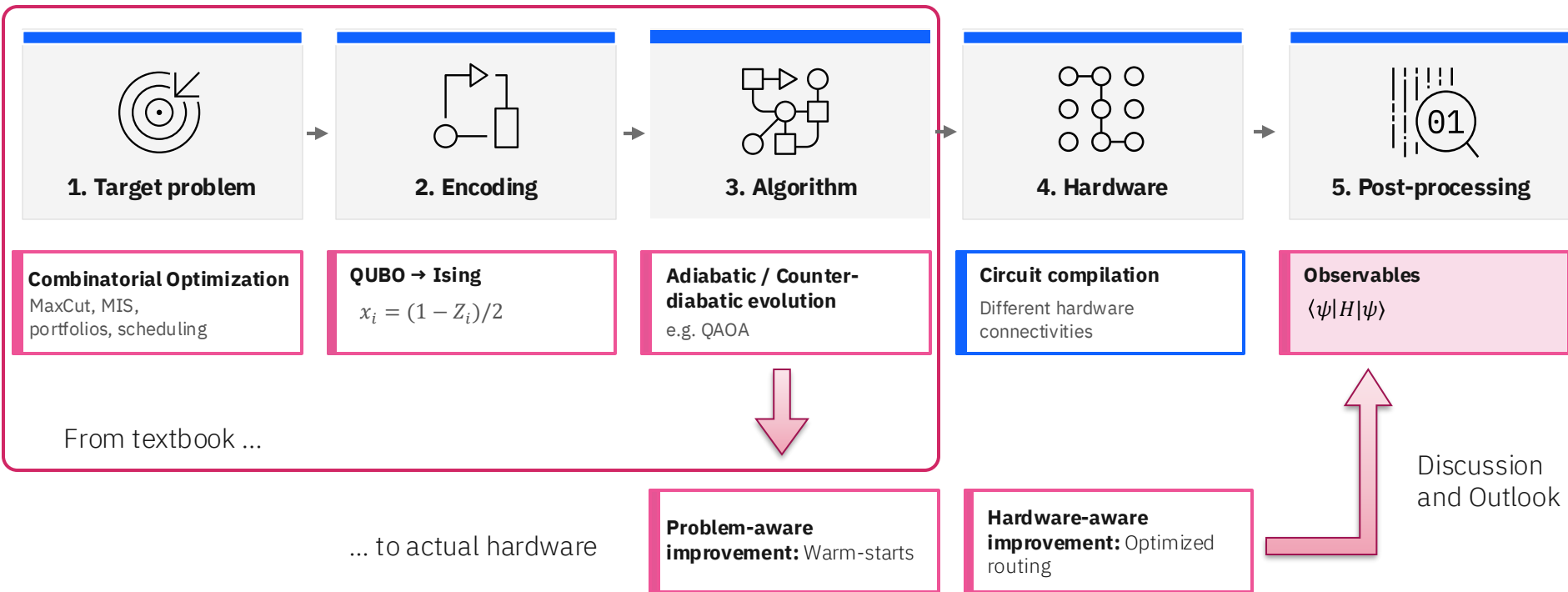
```
def train_parameters(objective_op: SparsePauliOp, qaoa_layers: int):

    # — Depth 1: brute-force scan + refine —
    trainer1 = DepthOneScanTrainer(
        EfficientDepthOneEvaluator())
    trainer2 = ScipyTrainer(...)
    result1 = trainer1.train(objective_op, ...)
    result2 = trainer2.train(
        objective_op, params0=result1[...])
    if qaoa_layers == 1: return result2

    # — Depth 2: transition states —
    sub_trainer1 = ScipyTrainer(LightConeEvaluator(), ...)
    trainer3 = TransitionStatesTrainer(sub_trainer1)
    result3 = trainer3.train(objective_op,
        previous_optimal_point=result2[...])
    if qaoa_layers == 2: return result3

    # — Depth 3: MPS recursion —
    sub_trainer2 = ScipyTrainer(
        MPSEvaluator(bond_dim_circuit=64, ...))
    trainer4 = RecursionTrainer(
        sub_trainer2, parameter_extender=interpolate)
    result4 = trainer4.train(
        objective_op, params0=result3[...], reps=3)
    if qaoa_layers == 3: return result4
```

Today's plan



Some recent QAOA experiments on superconducting qubits

What do we have?

Large scale experiments

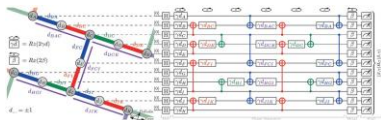
(beyond exact simulation of QAOA circuit)

What's missing?

Utility scale experiments

(beyond best-guess classical solvers)

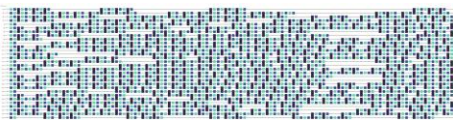
Quantum Annealing vs. QAOA:
127 Qubit Higher-Order Ising
Problems on NISQ Computers



What: 127 qubits on HW native
3rd order problems with $p = 2$.

Pelofske et al. (2023)
ISC High Performance 2023

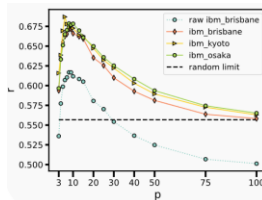
Large-scale QAOA on non-planar
graphs with machine learning
noise mitigation.



What: up to 40 qubits on RR3
graphs and depth $p = 2$.

Sack et al. (2024) Phys. Rev.
Research

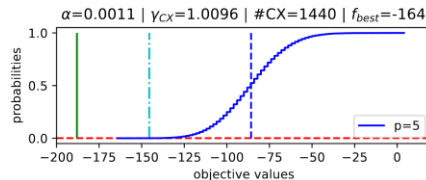
Towards a universal QAOA
protocol: Evidence of quantum
advantage in solving combinatorial
optimization problems.



What: up to 105 qubits on HW
native problems with p up to 100.

Montanez-Barrera et al.
arxiv:2405.09169

Provable bounds for noise-free
expectation values computed
from noisy samples.

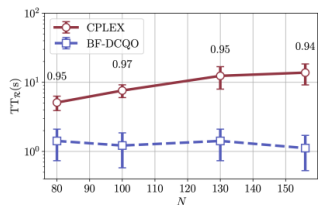


What: up to 127 qubits on HW
native 3rd order with $p = 5$.

Barron et al. arXiv:2312.00733

Some *even more recent* optimization experiments on superconducting qubits

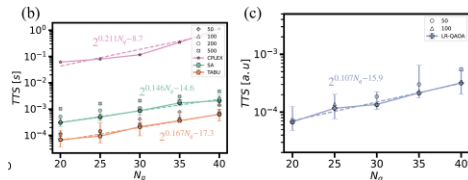
Runtime Quantum Advantage with Digital Quantum Optimization



What: 156 qubits on non-HW native 4rd order problems

Chandarana et al. (2025), arxiv:2505.8663v1

Toward a Linear-Ramp QAOA: evidence of a scaling advantage on combinatorial optimization problems



What: 109 qubits, depth $p = 100$, 21,200 CNOTs. Scaling advantage

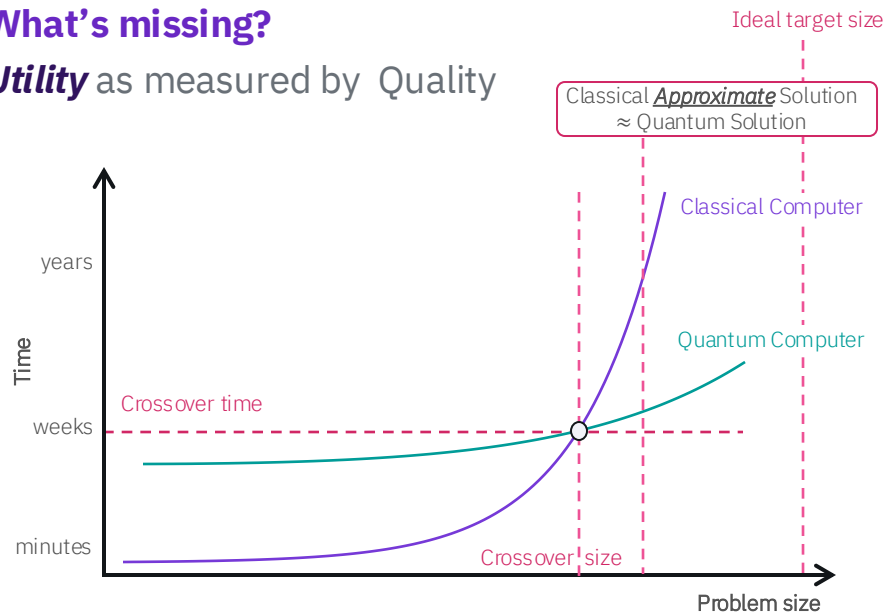
Montanez-Barrera et al. (2025) npj Quantum Inf. 11, 131.

What do we have?

Utility as measured by Resources

What's missing?

Utility as measured by Quality



TWO METRICS

1

Solution quality

2

Resources used, e.g. Time-To-Solution

Course Summary

Quantum Algorithm Landscape

- Few foundational ideas, many variants targeting error mitigation
- Many steps in the pipeline can use algorithmic improvement
- But as we get closer to a fault-tolerant era, we need more algorithms

Combinatorial Optimization

- Can easily convert Quadratic Programs into QUBOs
- Which we can then map to the ground-state problem to solve them with quantum tools

The Adiabatic Theorem and QAOA

- QAOA inherits converge guarantee from the Adiabatic Theorem
- But only in the deep circuit limit
- Meaning it is still accessible in the near-term, but has decreased solution quality

State-of-the-art experiments: e.g. Warm-starts and Efficient circuit compilation

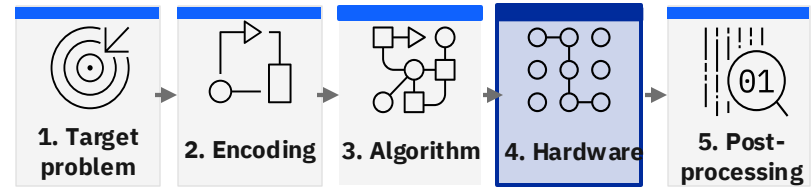
- Large scale experiments enabled by many error mitigation methods
- Utility terms of Time-of-Solution proved
- When/if will we have utility in terms of solution quality?



IBM Quantum

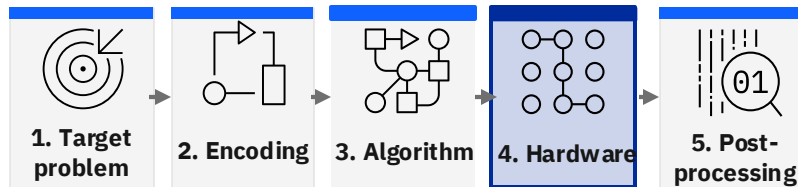


Transpilation for circuits with commuting gates



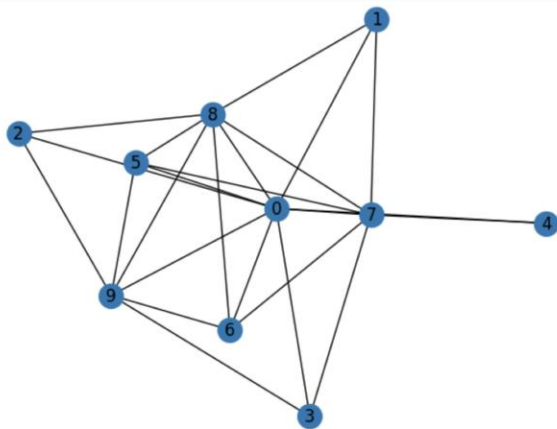
How to use the SWAP strategies approach through Qiskit

Transpilation for circuits with commuting gates



General approach vs SWAP strategies approach

Graph with 10 nodes
and 24 edges (density ≈ 0.53)



QAOA depth $p=3$

```
naive_count = naively_transpiled_qaoa.count_ops().get("cz", 0)
optimal_count = optimally_transpiled_qaoa.count_ops().get("cz", 0)

print(f"2q gate count for naive circuit = {naive_count}")
print(f"2q gate count for optimal circuit = {optimal_count}")
```

2q gate count for naive circuit = 359
2q gate count for optimal circuit = 333

```
naive_2q_depth = naively_transpiled_qaoa.depth(filter_function=lambda x: x.operation.name == "cz")
optimal_2q_depth = optimally_transpiled_qaoa.depth(
    filter_function=lambda x: x.operation.name == "cz"
)

print(f"2q depth for naive circuit = {naive_2q_depth}")
print(f"2q depth for optimal circuit = {optimal_2q_depth}")
```

2q depth for naive circuit = 241
2q depth for optimal circuit = 84

We learnt about

- Combinatorial optimization and how to formulate the problem as a ground state problem
- Adiabatic computing and how to derive the QAOA from it
- Extensions of QAOA and considerations when running on hardware.

Concluding thoughts

*Many optimization problems are NP-hard. However, many **classical approaches are heuristic** and work well in practice. Quantum approaches to combinatorial optimization, typically also heuristics, must be explored on practically valuable instances where classical heuristics struggle.*

*The real innovation happens where math insights meet hardware constraints — **problem-specific variants, approximations, and compilation tricks.***

Things to update

QOBLIB.

Repos: Qiskit Addon Optimization Mapper, QAOA training pipeline, and best practices.

QAOA alternatives, maybe consider Bias-Field QAOA.

Few Foundational Algorithmic Ideas, Many Variants

IBM Quantum

HARDWARE PRESSURE – LONG TERM

General goal: correct errors

Resulting algorithms:

- Surface codes
- LDPC codes
- Bicycle-bivariate codes
- ...

HARDWARE PRESSURE – SHORT TERM

General goal: reduce errors

Resulting algorithms:

- VQE
- Adapt-VQE
- RQAOA
- ...

Problem-based robustness:

- Warm-starts
- Problem-dependent mixers
- Approximate Implementations of higher-order problems
- Efficient Transpilation of QAOA circuits
- ...

Error mitigation:

- Probabilistic Error Cancellation
- Zero Noise Extrapolation
- ...

PHYSICS-INSPIRED

Adiabatic and Counter-Diabatic Evolution

General goal: Resolve energy spectrum of Hamiltonians.

Resulting algorithms:

- QAOA
- DC-QAOA / BF-DCQO
- Floquet Engineering

MATH-INSPIRED

Quantum Fourier Transform

General goal: Solve discrete problems.

Resulting algorithms:

- Phase estimation
- Shor's
- HHL

CS-INSPIRED

Quantum Oracles & Machine Learning

General goal: Build tools for quantum solvers.

Resulting algorithms:

- Grover's Search
- Amplitude Amplification
- DQI
- Quantum Neural Networks

Course outline

Part 1: Target problem classes

Part 2: Problem Encoding methods

Part 3: A foundational algorithm for quantum optimization

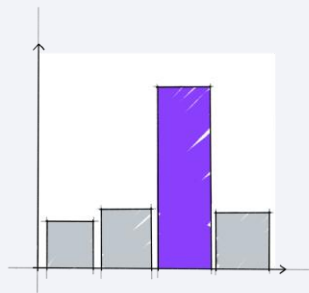
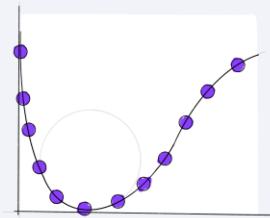
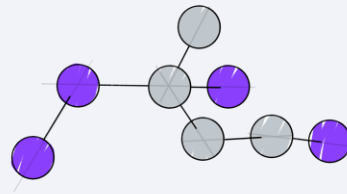
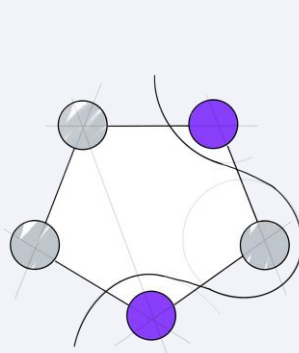
- The Adiabatic Theorem and Adiabatic Annealing
- From Adiabatic Annealing to QAOA

Part 4: Executing QAOA on Superconducting Qubits

Part 5: Recent Results and Future Prospects

Part 6: Discovering New Quantum Algorithms

- Fixing issues in existing implementations (full pipeline)
- Finding new target applications



Few Foundational Algorithmic Ideas, Many Variants

IBM Quantum

HARDWARE PRESSURE – LONG TERM

General goal: correct errors

Resulting algorithms:

- Surface codes
- LDPC codes
- Bicycle-bivariate codes
- ...

HARDWARE PRESSURE – SHORT TERM

General goal: reduce errors

Resulting algorithms:

- VQE
- Adapt-VQE
- RQAOA
- ...

Problem-based robustness:

- Warm-starts
- Problem-dependent mixers
- Approximate Implementations of higher-order problems
- Efficient Transpilation of QAOA circuits
- ...

Error mitigation:

- Probabilistic Error Cancellation
- Zero Noise Extrapolation
- ...

PHYSICS-INSPIRED

Adiabatic and Counter-Diabatic Evolution

General goal: Resolve energy spectrum of Hamiltonians.

Resulting algorithms:

- QAOA
- DC-QAOA / BF-DCQO
- Floquet Engineering

MATH-INSPIRED

Quantum Fourier Transform

General goal: Solve discrete problems.

Resulting algorithms:

- Phase estimation
- Shor's
- HHL

CS-INSPIRED

Quantum Oracles & Machine Learning

General goal: Build tools for quantum solvers.

Resulting algorithms:

- Grover's Search
- Amplitude Amplification
- DQI
- Quantum Neural Networks